

Auto Autónomo a Escala

Documentación técnica completa del pipeline de *Imagen* $\rightarrow$ *Acción*

Proyecto Integrador — Inteligencia Artificial (ENIB)

June 14, 2026

Abstract

Este documento explica, **de lo general a lo particular y función por función**, todo el software del proyecto: el modelo aprende a manejar un auto a escala prediciendo, a partir de una sola imagen de su cámara frontal, qué hacer con cada motor. Se cubre el panorama conceptual (qué se quiere, qué hardware hay, qué datos tenemos y sus limitaciones), el **pipeline de datos** (de los `labels.csv` crudos a los CSV que consume la red), los **cuatro motores de modelo** (per-frame, temporal, con signo e híbrido), los **entypoints** que congelan cada experimento, y la **herramienta de exploración de hiperparámetros**. Cada bloque incluye el código fuente completo y una explicación detallada. El hilo conductor del proyecto es una conclusión honesta: *con los datos disponibles, ninguna arquitectura sofisticada supera a un modelo per-frame simple; el techo es el dato.*

Contents

I	Contexto: qué problema resolvemos	2
1	El objetivo en una frase	2
2	El hardware y la física del problema	2
3	El dataset y sus limitaciones (lo más importante para interpretar resultados)	3
II	El pipeline de datos	3
4	<code>config.py</code> — el panel de control central	3
5	<code>build_global_csv.py</code> — de los <code>labels.csv</code> crudos al dataset	8
6	<code>build_signed_csv.py</code> — la variante con velocidad “con signo”	14
7	Herramientas de inspección	17
7.1	<code>data_sanity.py</code> — el chequeo de salud del dataset	17
7.2	<code>augmentation_tuner.py</code> — visualizar el color jitter	20
III	Los motores de modelo	21
8	<code>nn_perframe.py</code> — el modelo base (per-frame)	21
9	<code>nn_sequence.py</code> — el modelo temporal (CNN + GRU/LSTM)	30

10	<code>nn_signed.py</code> — velocidad con signo (el comando físico)	36
11	<code>nn_hybrid.py</code> — la solución: signed + cabeza “is-stop”	43
IV	Entrypoints y experimentación	49
12	La convención <code>model_NN_*</code>: un archivo por modelo	49
12.1	El baseline no-deep: <code>model_00_baseline.py</code>	50
12.2	El campeón: <code>model_01_perframe.py</code>	52
12.3	El híbrido: <code>model_09_hybrid.py</code>	52
12.4	Los diez modelos de un vistazo	53
13	<code>sweep.py</code> — exploración de hiperparámetros	53
V	Resultados y conclusión	60
14	El scoreboard	60
15	Conclusión: el arco completo	61

Part I

Contexto: qué problema resolvemos

1 El objetivo en una frase

Dada una **imagen** de la cámara frontal de un auto a escala, queremos predecir la **acción de manejo**: qué hacer con el motor izquierdo y con el derecho. Es **aprendizaje supervisado** (*imagen* → *acción*), un laboratorio de visión por computadora + toma de decisiones. **No es aprendizaje por refuerzo**: no hay recompensas ni exploración; tenemos grabaciones de un piloto humano manejando, y el modelo aprende a imitarlo.

El marco mental

Pensá en cada frame como una pregunta tipo examen: “dada esta foto, ¿qué hizo el piloto?”. La respuesta correcta (la etiqueta) es el comando que el piloto envió a los motores en ese instante. Entrenar = mostrarle miles de pares (foto, comando) y ajustar la red para que sus predicciones se parezcan a lo que hizo el humano.

2 El hardware y la física del problema

El auto es una Raspberry Pi 4 + un driver de motores **L298N** + una webcam. **No hay encoders ni sensores de velocidad**: lo único que se registra es lo que el piloto *ordenó*, no lo que el auto *midió*. Esto es central para entender las etiquetas.

- **speedA / speedB**: la consigna **PWM (0–100%)** al pin ENABLE del motor izquierdo (A) y derecho (B). Es un *comando*, no una medición de cuán rápido gira la rueda.
- **GPIO1..4**: los pines de sentido del puente H. La convención es **asimétrica** entre motores (un detalle de cableado que el código tiene que respetar):
 - Motor IZQ (A): (GPIO1,GPIO2)=(1,0) es adelante, (0,1) es atrás.
 - Motor DER (B): (GPIO3,GPIO4)=(0,1) es adelante, (1,0) es atrás.
- **Regla de freno**: el L298N *frena* el motor si los 4 GPIO de sentido están en 0. Por eso una fila con los 4 GPIO en 0 significa motor detenido; si además tuviera velocidad $\neq 0$, es un error y hay que forzar **speedA=speedB=0**.
- **Los giros son por PIVOTE de velocidad, no por GPIO**: para doblar a la izquierda, se pone la velocidad del motor izquierdo en 0 mientras el derecho sigue (¡el GPIO del motor parado puede seguir marcando “adelante”!). **Conclusión práctica: para saber qué hace un motor hay que mirar velocidad Y GPIO juntos**. Un motor con velocidad 0 está parado aunque su GPIO indique un sentido.

Por qué descomponemos por motor

El comportamiento “global” del auto (adelante, atrás, izquierda, derecha, stop) se puede reconstruir, pero es muy desbalanceado (casi no hay giros a la derecha). En cambio, si describimos la acción **por motor** (¿qué hace el izquierdo? ¿qué hace el derecho?), el problema se vuelve mucho más balanceado y aprendible. Esta decisión atraviesa todo el código.

3 El dataset y sus limitaciones (lo más importante para interpretar resultados)

Tenemos 3 sesiones de grabación, ~25.000 imágenes (<timestamp>.png) y unos 2.859 comandos de control en archivos `labels.csv` (separador ';'). O sea: ~9 imágenes por cada comando. Las limitaciones son serias y honestas, y explican por qué los resultados tienen el techo que tienen:

- **El 97% de los datos son UNA sola sesión.** Difícil medir generalización: el modelo entrena casi todo con un único recorrido.
- **Casi no existen giros a la derecha** (~16:1 a favor de la izquierda). Es real, no un bug. Se mitiga con *flip horizontal* (espejar la imagen convierte izquierda en derecha).
- **El 91% de las filas son interpoladas.** Solo ~9% son comandos reales; el resto se rellena entre comando y comando. Sumado a ~22 fps, hay mucha redundancia de frames casi idénticos.
- **El split debe ser por sesión, no aleatorio.** Si mezcláramos al azar, frames casi iguales caerían en train y val a la vez (fuga de datos) y el resultado sería optimista y falso.

El techo es el dato

A lo largo del proyecto probamos modelos cada vez más sofisticados. Spoiler: ninguno supera de forma real a un modelo per-frame simple. La razón no es la arquitectura sino estas limitaciones de datos. Tenerlo presente desde el principio ayuda a leer todo lo que sigue con la expectativa correcta.

Part II

El pipeline de datos

4 `config.py` — el panel de control central

Todo lo configurable vive en un único lugar: la dataclass `Config` (instancia `CFG`). Cualquier otro programa hace `from config import CFG` y lee de ahí. La filosofía es **no hardcodear parámetros sueltos**: si querés cambiar el learning rate, el dropout o cómo se balancean las clases, lo tocás acá. Correr `python config.py` genera además `augmentation_preview.png`, un gráfico que muestra cómo quedaría la distribución del dataset *antes* y *después* de aplicar los tratamientos (flip, remuestreo), para tunear a ojo sin entrenar.

```

1  """
2  config.py -- PANEL DE CONTROL CENTRAL del proyecto.
3  =====
4  Todo lo configurable vive aca. El resto de los programas importan 'CFG' y usan
5  estos valores (NO hardcodear nada de esto en otro lado).
6
7  from config import CFG
8  CFG.lr, CFG.batch_size, CFG.resample_factor, ...
9
10 Corre este archivo directo para VER el efecto de tus parametros sobre la
11 distribucion del dataset (antes vs despues de los tratamientos):
12
13 .venv/bin/python config.py          -> genera 'augmentation_preview.png'
14

```

```

15 Asi tocas un numero aca, re-corres, y ves en el grafico como cambia el balance.
16 """
17
18 from dataclasses import dataclass, field
19
20
21 @dataclass
22 class Config:
23     # ===== DATOS / RUTAS =====
24     train_csv: str = "dataset_train.csv"
25     val_csv: str = "dataset_val.csv"
26     image_size: int = 224
27
28     # ===== SINTESIS / AUGMENTATION =====
29     # El flip horizontal espeja la imagen y hace SWAP motor A<->B:
30     # - convierte LEFT<->RIGHT (balancea giros)
31     # - FORWARD / BACKWARD / STOP NO cambian de etiqueta (son simetricos)
32     # Probabilidad de aplicar flip, separada por tipo de frame:
33     flip_prob_turns: float = 0.5 # frames LEFT/RIGHT (ej: subir a 0.6 para
    ↪ balancear izq/der)
34     flip_prob_straight: float = 0.5 # frames FORWARD/BACKWARD/STOP (solo variedad
    ↪ visual)
35
36     # Factor de remuestreo por clase GLOBAL (cuantas muestras sinteticas generar):
37     # 0.0 = eliminar la clase | <1 = submuestrear | 1 = igual | >1 = sobremuestrear
38     # Ej: para "eliminar stop" -> "STOP": 0.0 ; para "aumentar stop" -> "STOP": 2.0
39     resample_factor: dict = field(default_factory=lambda: {
40         "FORWARD": 1.0, "LEFT": 1.0, "RIGHT": 1.0, "STOP": 1.0, "BACKWARD": 1.0,
41     })
42
43     # Color jitter (robustez a iluminacion)
44     jitter_brightness: float = 0.15
45     jitter_contrast: float = 0.15
46     jitter_saturation: float = 0.15
47     jitter_hue: float = 0.05
48
49     # ===== PERDIDA / BALANCEO =====
50     use_class_weights: bool = True # pesos inversos a la frecuencia en la CE de
    ↪ direccion
51     # Suaviza esos pesos elevandolos a esta potencia: 1.0 = inverso a freq (extremo),
52     # 0.5 = raiz cuadrada (modera la clase rara: backward ~6x -> ~2.5x), 0.0 =
    ↪ uniforme.
53     class_weight_power: float = 1.0
54     lambda_speed: float = 1.0 # peso de la regresion de velocidad en la loss
    ↪ total
55     lambda_stop: float = 1.0 # (modelo hibrido) peso de la BCE de la cabeza
    ↪ "is-stop"
56     stop_pos_weight_cap: float = 0.0 # (hibrido) tope al pos_weight de la cabeza
    ↪ is-stop (0=sin tope)
57
58     # ===== ARQUITECTURA DEL MODELO =====
59     backbone_frozen: bool = True # congelar el feature extractor de MobileNetV3
60     # Cuello compartido: una entrada por capa oculta. no de capas = len(lista);
    ↪ neuronas = valores.
61     # [256] -> 1 capa de 256
62     # [512, 256] -> 2 capas (512 y 256)
63     neck_hidden: list = field(default_factory=lambda: [256])
64     dropout: float = 0.4
65

```

```

66 # ===== OPTIMIZACION =====
67 batch_size: int = 64
68 lr: float = 1e-3
69 weight_decay: float = 1e-4
70 max_epochs: int = 15
71 scheduler_step: int = 7
72 scheduler_gamma: float = 0.1
73 num_workers: int = 4
74 seed: int = 42
75
76 # ===== MODELO SECUENCIAL (CNN + GRU/LSTM) =====
77 # Lo usa train_gru.py. El backbone (MobileNetV3) va SIEMPRE congelado:
78 # lo que se entrena es la red recurrente + cuello + 4 cabezas.
79 # La ventana es MANY-TO-ONE: se predice la accion del ULTIMO frame de la
80 # secuencia (causal y deployable: en el auto seria un buffer rodante).
81 # seq_len y seq_stride son ADEMAS el experimento de "frecuencia de muestreo"
82 # que pide el enunciado: a ~22fps, stride=3 ~= ~1s de contexto temporal.
83 seq_len: int = 8 # no de frames por secuencia (ventana temporal)
84 seq_stride: int = 3 # paso entre frames de la ventana (1 = consecutivos)
85 seq_window_step: int = 1 # anti-redundancia: subsampla ventanas en TRAIN
86 ↪ (1=todas; 4=1 de cada 4)
87 rnn_type: str = "GRU" # "GRU" o "LSTM"
88 rnn_hidden: int = 128 # tamano del estado oculto recurrente
89 rnn_layers: int = 1 # no de capas recurrentes apiladas
90
91 CFG = Config()
92
93
94 # =====
95 # Resumen de parametros activos (para anotar en los graficos)
96 # =====
97 def active_params_text(cfg=CFG):
98     rf = ", ".join(f"{k}:{v:g}" for k, v in cfg.resample_factor.items())
99     return (
100         f"flip_turns={cfg.flip_prob_turns:g} | "
101         ↪ f"flip_straight={cfg.flip_prob_straight:g} | "
102         f"class_weights={cfg.use_class_weights}\n"
103         f"resample_factor -> {rf}"
104     )
105
106 # =====
107 # Simulacion de la distribucion resultante (valor esperado)
108 # =====
109 def simulate_distribution(df, cfg=CFG):
110     """Calcula la distribucion de clases ANTES y DESPUES de aplicar:
111     (1) resample_factor por clase global, (2) flip estocastico (swap A<->B y L<->R).
112     Devuelve un dict con porcentajes por vista: global / motorA / motorB."""
113     import pandas as pd
114
115     flipmap = {"LEFT": "RIGHT", "RIGHT": "LEFT"}
116     grp = df.groupby(["behavior", "behaviorA",
117         ↪ "behaviorB"]).size().reset_index(name="c")
118
119     rows = []
120     for _, r in grp.iterrows():
121         gb, a, b, c = r["behavior"], r["behaviorA"], r["behaviorB"], r["c"]
122         w = cfg.resample_factor.get(gb, 1.0) * c

```

```

122     p = cfg.flip_prob_turns if gb in ("LEFT", "RIGHT") else cfg.flip_prob_straight
123     rows.append((gb, a, b, w * (1.0 - p))) # no flipeado
124     rows.append((flipmap.get(gb, gb), b, a, w * p)) # flipeado: swap motores y
    ↪ L<->R
125     sim = pd.DataFrame(rows, columns=["behavior", "behaviorA", "behaviorB", "w"])
126
127     g_order = ["FORWARD", "LEFT", "RIGHT", "STOP", "BACKWARD"]
128     m_order = ["STOP", "FORWARD", "BACKWARD"]
129
130     def before(col, order):
131         vc = df[col].value_counts(normalize=True).mul(100)
132         return [float(vc.get(k, 0.0)) for k in order]
133
134     def after(col, order):
135         s = sim.groupby(col)["w"].sum()
136         tot = s.sum()
137         return [float(s.get(k, 0.0) / tot * 100) if tot > 0 else 0.0 for k in order]
138
139     return {
140         "global": (g_order, before("behavior", g_order), after("behavior", g_order)),
141         "motorA": (m_order, before("behaviorA", m_order), after("behaviorA",
    ↪ m_order)),
142         "motorB": (m_order, before("behaviorB", m_order), after("behaviorB",
    ↪ m_order)),
143     }
144
145
146 def plot_preview(cfg=CFG, out_png="augmentation_preview.png"):
147     import os
148     import numpy as np
149     import pandas as pd
150     import matplotlib.pyplot as plt
151
152     if not os.path.exists(cfg.train_csv):
153         print(f"[ERROR] No existe {cfg.train_csv}. Corre build_global_csv.py
    ↪ primero.")
154         return
155     df = pd.read_csv(cfg.train_csv)
156     dist = simulate_distribution(df, cfg)
157
158     fig, axes = plt.subplots(1, 3, figsize=(20, 7))
159     fig.suptitle("Distribucion del dataset de ENTRENAMIENTO: ANTES vs DESPUES de los
    ↪ tratamientos",
160                 fontsize=15)
161     titles = {"global": "Behavior GLOBAL (5 clases)",
162              "motorA": "Direccion Motor A (IZQ)",
163              "motorB": "Direccion Motor B (DER)"}
164
165     for ax, key in zip(axes, ["global", "motorA", "motorB"]):
166         order, before, after = dist[key]
167         x = np.arange(len(order))
168         ax.bar(x - 0.2, before, 0.4, label="Antes (crudo)", color="#4C72B0")
169         ax.bar(x + 0.2, after, 0.4, label="Despues (config)", color="#DD8452")
170         for xi, (bv, av) in enumerate(zip(before, after)):
171             ax.text(xi - 0.2, bv + 0.5, f"{bv:.1f}", ha="center", va="bottom",
    ↪ fontsize=8)
172             ax.text(xi + 0.2, av + 0.5, f"{av:.1f}", ha="center", va="bottom",
    ↪ fontsize=8)
173             ax.set_xticks(x); ax.set_xticklabels(order, rotation=30)

```

```

174     ax.set_title(titles[key]); ax.set_ylabel("% del split"); ax.legend()
175     ax.grid(True, axis="y", alpha=0.3)
176
177     # Anotacion: QUE valores de las variables produjeron estos resultados.
178     fig.text(0.5, 0.005, "Parametros activos: " +
179     ↪ active_params_text(cfg).replace("\n", " | "),
180           ha="center", fontsize=10, family="monospace",
181           bbox=dict(boxstyle="round", facecolor="#f0f0f0", edgecolor="gray"))
182     plt.tight_layout(rect=[0, 0.05, 1, 1])
183     plt.savefig(out_png, dpi=150)
184     print(f"[graf] Preview guardado en '{out_png}'")
185     print("Parametros activos:\n" + active_params_text(cfg))
186
187 if __name__ == "__main__":
188     plot_preview()

```

Listing 1: config.py — esquema de todos los hiperparametros

Recorrido por los campos

- **Datos y rutas** (`train_csv`, `val_csv`, `image_size`): qué CSV consume el modelo y a qué resolución se llevan las imágenes (224, el tamaño nativo de MobileNet).
- **Síntesis / augmentation**: `flip_prob_turns` y `flip_prob_straight` controlan con qué probabilidad se espeja cada tipo de frame; `resample_factor` es un diccionario por clase global donde 0 elimina, <1 submuestra, 1 deja igual y >1 sobremuestra; los `jitter_*` controlan las perturbaciones de color (robustez a la iluminación).
- **Pérdida / balanceo**: `use_class_weights` activa pesos inversos a la frecuencia; `class_weight_power` suaviza esos pesos (1 = inverso puro, 0.5 = raíz, 0 = uniforme); `lambda_speed` pondera la regresión de velocidad y `lambda_stop` pondera la cabeza “is-stop” del modelo híbrido; `stop_pos_weight_cap` le pone un tope a un peso interno (lo vemos en el híbrido).
- **Arquitectura**: `neck_hidden` es una *lista* (su largo = número de capas del cuello, sus valores = neuronas por capa), `dropout` y `backbone_frozen` (si se congela el extractor).
- **Optimización**: `lr`, `batch_size`, `max_epochs`, el scheduler, etc.
- **Modelo secuencial**: `seq_len` (frames por ventana), `seq_stride` (paso entre frames), `seq_window_step` (anti-redundancia), `rnn_type/rnn_hidden/rnn_layers`.

Las funciones de config.py

- `active_params_text(cfg)`: devuelve un string corto que resume los parámetros activos; se usa para *anotar* los gráficos (así cada figura dice con qué config se generó).
- `simulate_distribution(df, cfg)`: **sin entrenar**, calcula la distribución de clases que resultaría de aplicar (1) el `resample_factor` y (2) el flip estocástico. Para el flip modela el valor esperado: una fracción (1-p) queda sin flipear y una fracción p se flipea (que intercambia motor A↔B y LEFT↔RIGHT). Devuelve las distribuciones global, motor A y motor B.
- `plot_preview(cfg)`: dibuja barras “antes vs después” para las tres vistas y las guarda en `augmentation_preview.png`, anotando los parámetros usados. Es la herramienta para *ver* el efecto de tocar un número antes de gastar horas de GPU.

5 build_global_csv.py — de los labels.csv crudos al dataset

Este es el corazón del pipeline de datos. Toma los labels.csv crudos (comandos dispersos en el tiempo) y las imágenes, y produce **dos CSV con una fila por imagen**: dataset_train.csv (la primera sesión, ~24.5k imágenes) y dataset_val.csv (las otras sesiones, ~686 imágenes). El split es **por sesión** a propósito (no aleatorio), para medir generalización honesta y evitar fuga de datos.

```

1  """
2  build_global_csv.py
3  =====
4  Genera un CSV global con UNA fila por imagen de todas las sesiones.
5
6  Columnas: las del labels.csv original + behavior, record y source:
7      time_in_ms , speedA , speedB , GPIO1 , GPIO2 , GPIO3 , GPIO4 , behavior , record
8      ↪ , source
9
10     - time_in_ms : timestamp de la IMAGEN (= nombre del archivo <ts>.png)
11     - behavior   : STOP / FORWARD / BACKWARD / LEFT / RIGHT
12     - record     : carpeta de sesion de la imagen
13     - source     : 'real' (comando real asociado) o 'interp' (comando interpolado)
14
15  LOGICA
16  -----
17  1) CORRECCION DE ERROR (freno): el driver L298N frena el motor si los 4 GPIO de
18     sentido estan en 0. Si una fila tiene los 4 GPIO en 0 y alguna velocidad != 0,
19     es un error: se fuerza speedA = speedB = 0.
20
21  2) ASOCIACION: cada comando se asocia a la imagen ANTERIOR mas cercana (ultima
22     imagen con timestamp <= tiempo del comando). Esa imagen toma los valores
23     exactos del comando.
24
25  3) RECORTE DE BORDES: el piloto dejaba el auto quieto largos periodos ANTES del
26     primer comando y DESPUES del ultimo. Esas imagenes se DESCARTAN.
27
28  4) INTERPOLACION: las imagenes sin comando asociado reciben velocidades interpoladas.
29     Los GPIO reciben la DIRECCION del lado que se esta moviendo.
30
31  5) GPIO EN INTERPOLADAS: si la velocidad interpolada es > 0, se mantiene la direccion.
32     Solo si la velocidad interpolada es 0 sus GPIO van a (0,0).
33
34  6) DECODIFICAR BEHAVIOR SIN UMBRAL: direccion decidida con velocidad + GPIO.
35
36  7) FILTRO DE LATENCIA EXCESIVA (NUEVO): Si la diferencia de tiempo entre dos frames
37     consecutivos es mayor a 1000ms, se descarta el frame para evitar alucinaciones
38     por saltos temporales/teletransportes y se generan en el csv distintas "sesiones"
39     ↪ de trabajo
40     (para respetar los tiempos de guardado originales, sino habria que meter un offset)
41  """
42
43  import argparse
44  import glob
45  import os
46  from bisect import bisect_left, bisect_right
47
48  import pandas as pd
49  from config import CFG
50
51  MAX_DELAY_MS = 1000 # Umbral maximo para considerar que hubo un salto/teletransporte

```

```

51
52 def apply_brake_rule(sa, sb, g1, g2, g3, g4):
53     """Si los 4 GPIO (redondeados) estan en 0 -> motor frenado -> velocidades a 0."""
54     if round(g1) == 0 and round(g2) == 0 and round(g3) == 0 and round(g4) == 0:
55         return 0.0, 0.0
56     return sa, sb
57
58 def decode_motor_A(sa, g1, g2):
59     """Direccion del motor IZQ (A) in {STOP, FORWARD, BACKWARD}. Convencion:
60     ↪ fwd=(1,0), bwd=(0,1)."""
61     if sa <= 0:
62         return "STOP"
63     G1, G2 = int(round(g1)), int(round(g2))
64     if (G1, G2) == (1, 0):
65         return "FORWARD"
66     if (G1, G2) == (0, 1):
67         return "BACKWARD"
68     return "STOP"
69
70 def decode_motor_B(sb, g3, g4):
71     """Direccion del motor DER (B) in {STOP, FORWARD, BACKWARD}. Convencion:
72     ↪ fwd=(0,1), bwd=(1,0)."""
73     if sb <= 0:
74         return "STOP"
75     G3, G4 = int(round(g3)), int(round(g4))
76     if (G3, G4) == (0, 1):
77         return "FORWARD"
78     if (G3, G4) == (1, 0):
79         return "BACKWARD"
80     return "STOP"
81
82 # Direccion por motor -> signo (+1 fwd, -1 bwd, 0 stop) para componer el behavior
83 ↪ global.
84 _DIR_SIGN = {"FORWARD": 1, "BACKWARD": -1, "STOP": 0}
85
86 def decode_behavior(sa, sb, g1, g2, g3, g4):
87     """Comportamiento GLOBAL de 5 clases a partir de las direcciones por motor, SIN
88     ↪ umbral.
89     Se usa SOLO para el flip LEFT<->RIGHT y para estadistica; el modelo no lo
90     ↪ consume."""
91     l = _DIR_SIGN[decode_motor_A(sa, g1, g2)]
92     r = _DIR_SIGN[decode_motor_B(sb, g3, g4)]
93     if l == 0 and r == 0:
94         return "STOP"
95     if l > 0 and r > 0:
96         return "FORWARD"
97     if l < 0 and r < 0:
98         return "BACKWARD"
99     return "RIGHT" if (l - r) > 0 else "LEFT"
100
101 def pick_dir(speed_i, sa_a, dir_a, sa_b, dir_b):
102     """Direccion (par de GPIO) de un motor en una imagen INTERPOLADA."""
103     if speed_i <= 0:
104         return (0, 0)
105     if sa_a > 0 and dir_a != (0, 0):
106         return dir_a
107     if sa_b > 0 and dir_b != (0, 0):
108         return dir_b
109     return (0, 0)

```

```

105
106 def interp_fields(t, cmds, cmd_times):
107     """Interpola los campos del comando en el tiempo t para una imagen sin comando."""
108     hi = bisect_left(cmd_times, t)
109     if hi == 0:
110         c = cmds[0]
111         return (round(c[1], 1), round(c[2], 1), c[3], c[4], c[5], c[6])
112     if hi >= len(cmds):
113         c = cmds[-1]
114         return (round(c[1], 1), round(c[2], 1), c[3], c[4], c[5], c[6])
115
116     a, b = cmds[hi - 1], cmds[hi]
117     ta, tb = a[0], b[0]
118     w = 0.0 if tb == ta else (t - ta) / (tb - ta)
119     sa = round(a[1] + (b[1] - a[1]) * w, 1)
120     sb = round(a[2] + (b[2] - a[2]) * w, 1)
121     g1, g2 = pick_dir(sa, a[1], (a[3], a[4]), b[1], (b[3], b[4]))
122     g3, g4 = pick_dir(sb, a[2], (a[5], a[6]), b[2], (b[5], b[6]))
123     return (sa, sb, g1, g2, g3, g4)
124
125 def process_record(record_dir, dataset_dir):
126     record = os.path.basename(record_dir)
127     images_dir = os.path.join(record_dir, "Images")
128     csv_path = os.path.join(record_dir, "labels.csv")
129     if not os.path.isdir(images_dir) or not os.path.isfile(csv_path):
130         return [], 0, 0, 0, 0
131
132     df = pd.read_csv(csv_path,
133         ↪ sep=";").sort_values("time_in_ms").reset_index(drop=True)
134     cmds = []
135     for _, row in df.iterrows():
136         sa, sb = float(row.speedA), float(row.speedB)
137         g1, g2, g3, g4 = int(row.GPIO1), int(row.GPIO2), int(row.GPIO3),
138         ↪ int(row.GPIO4)
139         sa, sb = apply_brake_rule(sa, sb, g1, g2, g3, g4)
140         cmds.append((float(row.time_in_ms), sa, sb, g1, g2, g3, g4))
141     cmd_times = [c[0] for c in cmds]
142
143     imgs = []
144     for fn in os.listdir(images_dir):
145         if fn.lower().endswith(".png"):
146             try:
147                 ts = int(os.path.splitext(fn)[0])
148             except ValueError:
149                 continue
150             imgs.append(ts)
151     imgs.sort()
152
153     if not imgs or not cmds:
154         return [], 0, 0, 0, 0
155
156     direct = {}
157     for c in cmds:
158         j = bisect_right(imgs, c[0]) - 1
159         if j >= 0:
160             direct[j] = c[1:]
161
162     if not direct:
163         return [], 0, 0, 0, 0

```

```

162 first_idx = min(direct)
163 last_idx = max(direct)
164 n_dropped_bordes = first_idx + (len(imgs) - 1 - last_idx)
165
166
167 rows = []
168 n_cmd = n_interp = n_dropped_delay = 0
169 last_ts = None
170 chunk_id = 1
171 current_record = f"{record}_seq{chunk_id}"
172
173 for i in range(first_idx, last_idx + 1):
174     ts = imgs[i]
175
176     # --- FILTRO DE LATENCIA Y SEGMENTACION DE SESIONES ---
177     if last_ts is not None and (ts - last_ts) > MAX_DELAY_MS:
178         n_dropped_delay += 1
179         chunk_id += 1
180         current_record = f"{record}_seq{chunk_id}" # Cortamos la serie temporal
181     ↪ aqui last_ts = ts # Seteamos la nueva base de tiempo para el siguiente frame
182     continue # Eliminamos el frame de transicion
183
184     last_ts = ts
185
186     if i in direct:
187         sa, sb, g1, g2, g3, g4 = direct[i]
188         source = "real"
189         n_cmd += 1
190     else:
191         sa, sb, g1, g2, g3, g4 = interp_fields(ts, cmds, cmd_times)
192         source = "interp"
193         n_interp += 1
194
195     sa, sb = apply_brake_rule(sa, sb, g1, g2, g3, g4)
196     # behaviorA/behaviorB se calculan AL FINAL, desde la velocidad+GPIO ya
197     ↪ procesados.
198     rows.append({
199         # --- columnas que CONSUME el modelo ---
200         "image_path": os.path.join(dataset_dir, record, "Images", f"{ts}.png"),
201         "speedA": round(sa, 1),
202         "speedB": round(sb, 1),
203         "behaviorA": decode_motor_A(sa, g1, g2),
204         "behaviorB": decode_motor_B(sb, g3, g4),
205         # --- columnas de DEBUG (GPIO reales/interpolados + behavior global) ---
206         "GPIO1": int(round(g1)),
207         "GPIO2": int(round(g2)),
208         "GPIO3": int(round(g3)),
209         "GPIO4": int(round(g4)),
210         "behavior": decode_behavior(sa, sb, g1, g2, g3, g4), # 5 clases, para el
211     ↪ flip L<->R
212         "time_in_ms": ts,
213         "record": current_record, # nombre de sesion segmentado
214         "source": source,
215     })
216
217     return rows, n_cmd, n_interp, n_dropped_bordes, n_dropped_delay
218
219 COLS = [

```

```

218 # Primeras 5: lo que consume el modelo (imagen -> speeds + direccion por motor)
219 "image_path", "speedA", "speedB", "behaviorA", "behaviorB",
220 # Debug: GPIO reales/interpolados, behavior global (para flip L<->R) y
    ↪ trazabilidad
221 "GPIO1", "GPIO2", "GPIO3", "GPIO4", "behavior", "time_in_ms", "record", "source",
222 ]
223
224
225 def build_split(record_dirs, dataset_dir):
226     """Procesa una lista de sesiones y devuelve (rows, dropped_bordes,
    ↪ dropped_delay)."""
227     all_rows = []
228     total_dropped_bordes = 0
229     total_dropped_delay = 0
230     for rd in record_dirs:
231         rows, n_cmd, n_interp, n_drop_bordes, n_drop_delay = process_record(rd,
    ↪ dataset_dir)
232         total_dropped_bordes += n_drop_bordes
233         total_dropped_delay += n_drop_delay
234         print(f" {os.path.basename(rd):30s} -> {len(rows):6d} imgs  "
235               f"(cmd: {n_cmd}, interp: {n_interp}, bordes desc.: {n_drop_bordes},
    ↪ delays desc.: {n_drop_delay}")
236         all_rows.extend(rows)
237     return all_rows, total_dropped_bordes, total_dropped_delay
238
239
240 def dump_split(rows, path, name, dropped_bordes, dropped_delay):
241     out = pd.DataFrame(rows, columns=COLS)
242     out.to_csv(path, index=False)
243     print("\n" + "=" * 60)
244     print(f"CSV {name}: {len(out)} filas -> {path}")
245     print(f"Imagenes descartadas en bordes (inactividad): {dropped_bordes}")
246     print(f"Imagenes descartadas por saltos (>1s): {dropped_delay}")
247     print("=" * 60)
248     if len(out):
249         print(f"-- Distribucion BEHAVIOR global {name} (%) --")
250         print((out["behavior"].value_counts(normalize=True) *
    ↪ 100).round(1).to_string())
251         print(f"-- Distribucion por motor {name} (%) --")
252         per_motor = pd.DataFrame({
253             "motorA": out["behaviorA"].value_counts(normalize=True) * 100,
254             "motorB": out["behaviorB"].value_counts(normalize=True) * 100,
255         }).round(1).fillna(0)
256         print(per_motor.to_string())
257
258
259 def main():
260     ap = argparse.ArgumentParser(
261         description="Genera CSVs imagen->comando con split por sesion (train/val)."
262     )
263     ap.add_argument("--dataset-dir", default="DataSet", help="carpeta raiz con los
    ↪ Record_*)")
264     ap.add_argument("--output-train", default=CFG.train_csv, help="CSV de
    ↪ entrenamiento")
265     ap.add_argument("--output-val", default=CFG.val_csv, help="CSV de validacion")
266     args = ap.parse_args()
267
268     record_dirs = sorted(
269         d for d in glob.glob(os.path.join(args.dataset_dir, "*"))

```

```

270         if os.path.isdir(d) and os.path.basename(d).lstrip("#").startswith("Record")
271     )
272     if not record_dirs:
273         print(f"No se encontraron carpetas Record_* en '{args.dataset_dir}'.")
274         return
275
276     # SPLIT POR SESION: la PRIMERA sesion (cronologica, la grande con >20k imagenes)
277     # se usa para ENTRENAR; el resto de las sesiones se reservan para VALIDAR.
278     train_dirs = record_dirs[:1]
279     val_dirs = record_dirs[1:]
280     if not val_dirs:
281         print("[ADVERTENCIA] Solo hay una sesion: no queda nada para validacion.")
282
283     print(f"Sesiones encontradas: {len(record_dirs)}")
284     print(f"  TRAIN: {' ', ' '.join(os.path.basename(d) for d in train_dirs)}")
285     print(f"  VAL  : {' ', ' '.join(os.path.basename(d) for d in val_dirs) or
    ↪  '(ninguna)'}")
286
287     print("\n[TRAIN]")
288     train_rows, tr_bordes, tr_delay = build_split(train_dirs, args.dataset_dir)
289     print("\n[VAL]")
290     val_rows, va_bordes, va_delay = build_split(val_dirs, args.dataset_dir)
291
292     dump_split(train_rows, args.output_train, "TRAIN", tr_bordes, tr_delay)
293     dump_split(val_rows, args.output_val, "VAL", va_bordes, va_delay)
294
295 if __name__ == "__main__":
296     main()

```

Listing 2: build_global_csv.py — asociacion, recorte, interpolacion y decodificacion

La lógica, paso a paso

1. `apply_brake_rule(sa, sb, g1..g4)`: implementa la regla física de freno. Si los 4 GPIO están en 0, fuerza las velocidades a 0. Se aplica dos veces: al leer cada comando y al final, para garantizar consistencia.
2. `decode_motor_A` y `decode_motor_B`: traducen velocidad + GPIO de un motor a su dirección $\in \{\text{STOP, FORWARD, BACKWARD}\}$. Clave: si la velocidad es ≤ 0 el motor está **STOP** sin importar el GPIO (recordá el pivote). Si se mueve, el par de GPIO decide el sentido, respetando la convención asimétrica.
3. `decode_behavior(...)`: compone el comportamiento *global* de 5 clases (STOP/FORWARD/BACKWARD/LEFT/RIGHT) a partir del signo de cada motor. Se usa **solo** para decidir el flip izquierda↔derecha y para estadística; el modelo no lo consume.
4. `pick_dir(...)` e `interp_fields(...)`: el motor de la **interpolación**. Para una imagen que no tiene comando propio, `interp_fields` busca con búsqueda binaria (**bisect**) el comando real anterior y el posterior, e interpola *linealmente por tiempo* las velocidades. Para la dirección (GPIO), **no** interpola el bit: `pick_dir` mantiene el sentido del lado que se está moviendo (un frenado conserva la dirección previa hasta llegar a 0). Esto evita que un frenado se decodifique erróneamente como STOP.
5. `process_record(...)`: orquesta todo para una sesión. Resumen del flujo:
 - Lee el `labels.csv`, aplica la regla de freno a cada comando, arma la lista `cmds`.
 - Lista los timestamps de las imágenes (el nombre del archivo *es* el timestamp).

- **Asociación:** a cada comando lo pega a la *imagen anterior más cercana* (la última con $\text{timestamp} \leq$ el del comando), usando `bisect_right - 1`. Si varios comandos caen entre dos frames, gana el último (de ahí la pérdida documentada de $\sim 22\%$ de comandos por colisión).
 - **Recorte de bordes:** descarta las imágenes anteriores al primer comando asociado y posteriores al último (el piloto dejaba el auto quieto largo rato antes y después; esos frames no son decisiones).
 - **Filtro de latencia y segmentación:** si entre dos frames consecutivos hay un salto > 1000 ms (`MAX_DELAY_MS`), descarta el frame de transición e incrementa un `chunk_id`, cortando la serie en sub-sesiones `_seqN`. Esto evita “teletransportes” temporales y, además, le da al modelo secuencial fronteras limpias donde no cruzar.
 - Para cada imagen del rango útil: si tiene comando, usa los valores reales (`source=real`); si no, interpola (`source=interp`). Re-aplica la regla de freno y, **al final**, decodifica `behaviorA/behaviorB` desde `velocidad+GPIO` ya procesados.
6. `build_split`, `dump_split`, `main`: `main` descubre las carpetas `Record_*`, manda la primera a `TRAIN` y el resto a `VAL`, procesa cada split y vuelca los CSV imprimiendo estadísticas (distribución global y por motor, descartes por borde y por salto). Las **primeras 5 columnas** del CSV son las que consume el modelo (`image_path`, `speedA`, `speedB`, `behaviorA`, $\hookrightarrow$ `behaviorB`); el resto es debug (`GPIO`, `behavior` global, tiempo, sesión, fuente).

Distribución por motor (train)

Motor A: STOP 48.5% / FWD 46.0% / BWD 5.5%. Motor B: FWD 81.2% / STOP 13.4% / BWD 5.5%. Al descomponer por motor, el desbalance brutal de `RIGHT` (0.4% global) desaparece: la clase rara pasa a ser `BACKWARD` (5.5%), mucho más aprendible. Este número reaparece en cada balanceo del proyecto.

6 `build_signed_csv.py` — la variante con velocidad “con signo”

Más adelante experimentamos con una representación distinta del target: en vez de (dirección + velocidad) por motor, **una sola variable continua con signo** (`PWM` $\in [-100, 100]$: positivo = adelante, negativo = atrás, 0 = parado). Este script genera `dataset_train_signed.csv` / `dataset_val_signed.csv`.

```

1  """
2  build_signed_csv.py
3  =====
4  Genera las bases SIGNED (train/val) para el experimento de VELOCIDAD CON SIGNO.
5
6  En vez de (direccion {STOP,FWD,BWD} + velocidad [0,100]) por motor, el target es UNA
7  variable continua por motor: el PWM con SIGNO in [-100, 100]:
8      > 0  -> FORWARD      < 0  -> BACKWARD      = 0  -> STOP
9
10 POR QUE ENVUELVE build_global_csv (y no reimplementa):
11 toda la logica delicada (regla de freno, asociacion comando->imagen anterior, recorte
12 de bordes, filtro de latencia >1s con segmentacion en _seqN, interpolacion + pick_dir,
13 decode por motor) YA esta validada ahi. Aca la reusamos tal cual y solo TRANSFORMAMOS
14 el target al final:
15     signed_speedA = speedA * signo(behaviorA)    (FWD=+1, BWD=-1, STOP=0)
16     signed_speedB = speedB * signo(behaviorB)
17 Como el decode garantiza STOP <=> velocidad 0, |signed| == speed y el signo es la

```



```

18  direccion. No se interpola "a traves del cero" (eso seria reimplementar la
    ↳ interpolacion);
19  la diferencia solo afecta reversas FWD->BWD sin STOP intermedio, casi inexistentes
    ↳ aca.
20
21  COLUMNAS DE SALIDA:
22      image_path, signed_speedA, signed_speedB,      <- lo que CONSUME el modelo
    ↳ (imagen->2 continuas)
23      behaviorA, behaviorB, behavior,                <- debug + flip L<->R / resample por
    ↳ clase
24      time_in_ms, record, source                    <- trazabilidad
25
26  NO toca dataset_train.csv / dataset_val.csv (las viejas siguen intactas).
27  """
28
29  import argparse
30  import glob
31  import os
32
33  import pandas as pd
34
35  # Reusamos TODA la maquinaria validada del builder original.
36  from build_global_csv import build_split, _DIR_SIGN
37
38  SIGNED_COLS = [
39      "image_path", "signed_speedA", "signed_speedB",    # consume el modelo
40      "behaviorA", "behaviorB", "behavior",              # debug + flip/resample
41      "time_in_ms", "record", "source",                  # trazabilidad
42  ]
43
44
45  def to_signed_rows(rows):
46      """Transforma las filas del builder original (speed+dir+GPIO) a target con
    ↳ signo."""
47      out = []
48      for r in rows:
49          sa = round(r["speedA"] * _DIR_SIGN[r["behaviorA"]], 1)
50          sb = round(r["speedB"] * _DIR_SIGN[r["behaviorB"]], 1)
51          out.append({
52              "image_path": r["image_path"],
53              "signed_speedA": sa,
54              "signed_speedB": sb,
55              "behaviorA": r["behaviorA"],
56              "behaviorB": r["behaviorB"],
57              "behavior": r["behavior"],
58              "time_in_ms": r["time_in_ms"],
59              "record": r["record"],
60              "source": r["source"],
61          })
62      return out
63
64
65  def dump_split(rows, path, name):
66      out = pd.DataFrame(rows, columns=SIGNED_COLS)
67      out.to_csv(path, index=False)
68      print("\n" + "=" * 60)
69      print(f"CSV SIGNED {name}: {len(out)} filas -> {path}")
70      print("=" * 60)
71      if not len(out):

```



```

72     return
73     # Verificacion: distribucion del signo por motor (debe espejar la direccion).
74     for col in ["signed_speedA", "signed_speedB"]:
75         v = out[col]
76         z = (v == 0).mean() * 100
77         p = (v > 0).mean() * 100
78         n = (v < 0).mean() * 100
79         mag = v[v != 0].abs()
80         print(f" {col:14s}: STOP(0) {z:5.1f}% | FWD(+) {p:5.1f}% | BWD(-) {n:5.1f}%"
81               f" | |v|!=0: min {mag.min():.0f} mean {mag.mean():.1f} max
82               ↪ {mag.max():.0f}")
83         print(f" source: {dict(out['source'].value_counts())}")
84         print(f" -- behavior global (%) --")
85         print((out["behavior"].value_counts(normalize=True) * 100).round(1).to_string())
86
87     def main():
88         ap = argparse.ArgumentParser(description="Genera CSVs SIGNED (imagen->PWM con
89         ↪ signo por motor).")
90         ap.add_argument("--dataset-dir", default="DataSet", help="carpetas raiz con los
91         ↪ Record_*.")
92         ap.add_argument("--output-train", default="dataset_train_signed.csv", help="CSV
93         ↪ de entrenamiento SIGNED")
94         ap.add_argument("--output-val", default="dataset_val_signed.csv", help="CSV de
95         ↪ validacion SIGNED")
96         args = ap.parse_args()
97
98         record_dirs = sorted(
99             d for d in glob.glob(os.path.join(args.dataset_dir, "*"))
100             if os.path.isdir(d) and os.path.basename(d).lstrip("#").startswith("Record")
101         )
102         if not record_dirs:
103             print(f"No se encontraron carpetas Record_* en '{args.dataset_dir}'.")
104             return
105
106         # MISMO split por sesion que build_global_csv: 1a sesion (la grande) -> TRAIN,
107         ↪ resto -> VAL.
108         train_dirs = record_dirs[:1]
109         val_dirs = record_dirs[1:]
110         print(f"Sesiones: {len(record_dirs)}")
111         print(f" TRAIN: {' ', ' '.join(os.path.basename(d) for d in train_dirs)}")
112         print(f" VAL : {' ', ' '.join(os.path.basename(d) for d in val_dirs) or
113               ↪ '(ninguna)'}")
114
115         print("\n[TRAIN]")
116         train_rows, _, _ = build_split(train_dirs, args.dataset_dir)
117         print("\n[VAL]")
118         val_rows, _, _ = build_split(val_dirs, args.dataset_dir)
119
120         dump_split(to_signed_rows(train_rows), args.output_train, "TRAIN")
121         dump_split(to_signed_rows(val_rows), args.output_val, "VAL")
122
123     if __name__ == "__main__":
124         main()

```

Listing 3: build_signed_csv.py — reusa el pipeline validado y transforma el target

La decisión de ingeniería clave fue *no reimplementar* la lógica delicada (freno, asociación,

recorte, interpolación). En vez de eso, este script **envuelve** a `build_global_csv`: importa su `build_split` ya validado, y en `to_signed_rows` solo transforma el target con `signed = speed × signo(behavior)`. Como el decodificador garantiza que STOP implica velocidad 0, el signo cae perfecto. `dump_split` guarda el CSV e imprime una verificación útil (qué porcentaje de cada motor quedó en 0/positivo/negativo). `main` replica el mismo split por sesión.

7 Herramientas de inspección

Dos scripts auxiliares, que no entrenan nada, sirven para *entender* los datos antes de modelar.

7.1 `data_sanity.py` — el chequeo de salud del dataset

```

1 import pandas as pd
2 import numpy as np
3 import os
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from config import CFG
8
9 def run_benchmark(train_csv=None, val_csv=None):
10     train_csv = train_csv or CFG.train_csv
11     val_csv = val_csv or CFG.val_csv
12     missing = [p for p in (train_csv, val_csv) if not os.path.exists(p)]
13     if missing:
14         print(f"[ERROR] No se encontraron los archivos: {'', '.join(missing)}".")
15         print("Corre primero 'build_global_csv.py' para generarlos.")
16         return
17
18     print("Cargando datasets (train + val)...")
19     df_train = pd.read_csv(train_csv)
20     df_val = pd.read_csv(val_csv)
21     df_train["split"] = "train"
22     df_val["split"] = "val"
23     df = pd.concat([df_train, df_val], ignore_index=True)
24
25     print("\n=====")
26     print("[graf] REPORTE DE DATA SANITY Y BENCHMARKING")
27     print("=====")
28     print(f"Split por sesion -> Train: {len(df_train)} frames | Val: {len(df_val)}")
29     print("↪ frames")
30
31     # 0. DISTRIBUCION DE CLASES POR SPLIT
32     print("\n--- DISTRIBUCION BEHAVIOR GLOBAL POR SPLIT (%) ---")
33     by_split = (
34         df.groupby("split")["behavior"].value_counts(normalize=True).mul(100).round(1)
35         .unstack(fill_value=0)
36     )
37     print(by_split.to_string())
38
39     # Ob. DISTRIBUCION POR MOTOR (lo que realmente predice el modelo)
40     print("\n--- DISTRIBUCION DIRECCION POR MOTOR Y SPLIT (%) ---")
41     for split in ["train", "val"]:
42         sub = df[df["split"] == split]
43         per_motor = pd.DataFrame({
44             "motorA": sub["behaviorA"].value_counts(normalize=True) * 100,
45             "motorB": sub["behaviorB"].value_counts(normalize=True) * 100,
46         }).round(1).fillna(0)

```

```

46     print(f"[{split}]")
47     print(per_motor.to_string())
48
49     # 1. VERIFICACION DE INTEGRIDAD BASICA
50     total_rows = len(df)
51     null_counts = df.isnull().sum().sum()
52     print(f"Total de registros: {total_rows}")
53     print(f"Valores nulos en el CSV: {null_counts} " + ("(PELIGRO!)" if null_counts >
    ↪ 0 else "(Ok)"))
54
55     # 2. VERIFICACION DE IMAGENES EN DISCO
56     print("\nVerificando existencia fisica de las imagenes...")
57     # Tomamos una muestra aleatoria para no saturar el disco si son 20k+ imagenes
58     sample_size = min(2000, total_rows)
59     missing_images = 0
60     for path in df['image_path'].sample(sample_size):
61         if not os.path.exists(path):
62             missing_images += 1
63
64     missing_ratio = missing_images / sample_size
65     print(f"Imagenes perdidas en disco (Muestra de {sample_size}): {missing_images}")
66     if missing_ratio > 0:
67         print(f"[ALERTA ROJA] Aproximadamente el {missing_ratio*100:.1f}% de las
    ↪ rutas estan rotas.")
68     else:
69         print("Integridad de rutas de imagen: (Ok)")
70
71     # 3. RATIO DE FUENTES (REAL VS INTERPOLADO)
72     print("\n--- DISTRIBUCION DE FUENTE DE COMANDOS ---")
73     source_dist = df['source'].value_counts(normalize=True) * 100
74     print(source_dist.round(2).astype(str) + " %")
75     if 'interp' in source_dist and source_dist['interp'] > 85:
76         print("[ADVERTENCIA] Tasa de interpolacion extremadamente alta. Cuidado con
    ↪ el overfitting al algoritmo.")
77
78     # 4. BALANCE DE COMPORTAMIENTO (BEHAVIOR)
79     print("\n--- DISTRIBUCION DE CLASES (BEHAVIOR) ---")
80     behavior_dist = df['behavior'].value_counts(normalize=True) * 100
81     print(behavior_dist.round(2).astype(str) + " %")
82
83     # 5. ANALISIS DE DELTA T (TIEMPO ENTRE FRAMES)
84     print("\n--- ANALISIS DE LATENCIA DE CAMARA (Delta T) ---")
85     df = df.sort_values(by=['record', 'time_in_ms'])
86     df['delta_t'] = df.groupby('record')['time_in_ms'].diff()
87
88     dt_mean = df['delta_t'].mean()
89     dt_median = df['delta_t'].median()
90     dt_max = df['delta_t'].max()
91     fps_aprox = 1000 / dt_median if dt_median > 0 else 0
92
93     print(f"Delta T Medio: {dt_mean:.2f} ms")
94     print(f"Delta T Mediana: {dt_median:.2f} ms (Aprox {fps_aprox:.1f} FPS)")
95     print(f"Delta T Maximo: {dt_max:.2f} ms")
96
97     saltos_criticos = len(df[df['delta_t'] > 1000])
98     print(f"Frames con delay > 1 segundo: {saltos_criticos}")
99
100    # 6. ESTADISTICAS DE PWM (SPEED A y B)
101    print("\n--- ESTADISTICAS DE MOTORES ---")

```

```

102 print("Velocidad Motor Izq (A):")
103 print(df['speedA'].describe()[['mean', 'std', 'min', '50%', 'max']].to_string())
104 print("Velocidad Motor Der (B):")
105 print(df['speedB'].describe()[['mean', 'std', 'min', '50%', 'max']].to_string())
106
107 # =====
108 # GENERACION DE GRAFICOS VISUALES
109 # =====
110 print("\nGenerando graficos de diagnostico ('benchmark_report.png')...")
111 sns.set_theme(style="whitegrid")
112 fig, axes = plt.subplots(2, 3, figsize=(22, 11))
113 fig.suptitle('Benchmark del Dataset de Conduccion Autonoma', fontsize=16)
114
115 dir_order = ["STOP", "FORWARD", "BACKWARD"]
116
117 def motor_pct(col):
118     """% de cada direccion DENTRO de cada split (para que val no quede
    ↪ aplastado)."""
119     return (df.groupby('split')[col].value_counts(normalize=True).mul(100)
120             .rename('pct').reset_index())
121
122 # Grafico 1: Behavior global por fuente (real/interp)
123 sns.countplot(data=df, x='behavior', order=df['behavior'].value_counts().index,
124               ax=axes[0, 0], hue='source')
125 axes[0, 0].set_title('Behavior GLOBAL (5 clases) por Fuente')
126 axes[0, 0].tick_params(axis='x', rotation=45)
127
128 # Grafico 2: Direccion Motor A (IZQ) por split
129 sns.barplot(data=motor_pct('behaviorA'), x='behaviorA', y='pct', hue='split',
130             order=dir_order, ax=axes[0, 1])
131 axes[0, 1].set_title('Direccion Motor A (IZQ) por Split')
132 axes[0, 1].set_xlabel('direccion'); axes[0, 1].set_ylabel('% dentro del split')
133
134 # Grafico 3: Direccion Motor B (DER) por split
135 sns.barplot(data=motor_pct('behaviorB'), x='behaviorB', y='pct', hue='split',
136             order=dir_order, ax=axes[0, 2])
137 axes[0, 2].set_title('Direccion Motor B (DER) por Split')
138 axes[0, 2].set_xlabel('direccion'); axes[0, 2].set_ylabel('% dentro del split')
139
140 # Grafico 4: Delta T
141 sns.histplot(data=df[df['delta_t'] < 2000], x='delta_t', bins=50, ax=axes[1, 0],
142              ↪ color='coral')
143 axes[1, 0].set_title('Distribucion de Latencia entre Frames (< 2s)')
144 axes[1, 0].set_xlabel('Delta T (ms)')
145
146 # Grafico 5: Densidad de velocidades PWM
147 sns.kdeplot(data=df, x='speedA', label='Motor Izq (A)', fill=True, ax=axes[1, 1])
148 sns.kdeplot(data=df, x='speedB', label='Motor Der (B)', fill=True, ax=axes[1, 1])
149 axes[1, 1].set_title('Densidad de Velocidades PWM')
150 axes[1, 1].legend()
151
152 # Grafico 6: Rastro de velocidades en la sesion con MAS frames (evita chunks seq
    ↪ de 1 fila)
153 muestra_record = df['record'].value_counts().idxmax()
154 df_sample = df[df['record'] ==
    ↪ muestra_record].sort_values('time_in_ms').reset_index().head(200)
155 sns.lineplot(data=df_sample, x=df_sample.index, y='speedA', label='Motor A',
    ↪ ax=axes[1, 2])

```

```

155     sns.lineplot(data=df_sample, x=df_sample.index, y='speedB', label='Motor B',
    ↪ ax=axes[1, 2])
156     axes[1, 2].set_title(f'Rastro de Velocidad (Sample: {muestra_record})')
157     axes[1, 2].set_xlabel('No de Frame')
158
159     plt.tight_layout()
160     plt.savefig('benchmark_report.png', dpi=300)
161     print("Listo! Analisis completo.")
162
163 if __name__ == "__main__":
164     run_benchmark()

```

Listing 4: data_sanity.py — integridad, distribuciones y graficos de diagnostico

`run_benchmark` carga ambos CSV, los junta marcando el split, y produce un reporte de texto + una figura de 6 paneles (`benchmark_report.png`). Verifica: distribución de clases por split (global y por motor), valores nulos, existencia física de una muestra de imágenes (rutas rotas), ratio real/interpolado (alerta si $> 85\%$), análisis del ΔT entre frames (de donde sale el ~ 22 fps y la cuenta de saltos > 1 s), y estadísticas de PWM por motor. Es la herramienta que respalda con números todas las “limitaciones del dataset” de la primera parte.

7.2 augmentation_tuner.py — visualizar el color jitter

```

1  import matplotlib.pyplot as plt
2  from PIL import Image
3  import torchvision.transforms as T
4  import sys
5  import os
6
7  def run_tuning_session(image_path):
8      if not os.path.exists(image_path):
9          print(f"[ERROR] No se encuentra la imagen: {image_path}")
10         return
11
12         # Cargar la imagen original en formato PIL RGB
13         original_img = Image.open(image_path).convert('RGB')
14
15         # =====
16         # HIPERPARAMETROS DE COLOR JITTER (Stress Test)
17         # =====
18         # Llevamos el brillo al +/- 70% y el contraste al +/- 60%
19         jitter_transform = T.ColorJitter(
20             brightness=0.7, # Puede oscurecer hasta casi negro o sobreexponer a casi
    ↪ blanco
21             contrast=0.6,   # Fuerza la diferencia entre la linea blanca y el piso gris
22             saturation=0.5, # Lava o satura agresivamente los colores
23             hue=0.05        # Mantenemos el matiz casi intacto para no volver violeta el
    ↪ asfalto
24         )
25
26         # Configurar el lienzo (Grid de 4x4)
27         fig, axes = plt.subplots(4, 4, figsize=(12, 10))
28         fig.suptitle('Color Jittering Tuning Session (PyTorch Pipeline)', fontsize=16)
29
30         # Posicion [0,0]: Imagen Original
31         axes[0, 0].imshow(original_img)
32         axes[0, 0].set_title("Original")
33         axes[0, 0].axis('off')

```

```

34
35     # Generar 15 variaciones estocasticas
36     for i in range(1, 16):
37         row = i // 4
38         col = i % 4
39
40         # Aplicar la transformacion aleatoria
41         aug_img = jitter_transform(original_img)
42
43         axes[row, col].imshow(aug_img)
44         axes[row, col].set_title(f"Variation {i}")
45         axes[row, col].axis('off')
46
47     plt.tight_layout()
48     plt.savefig('tuning_grid.png', dpi=300)
49     print("Grid generado: 'tuning_grid.png'. Abrelo y analiza las variaciones.")
50     plt.show()
51
52 if __name__ == "__main__":
53     # Puedes pasar la ruta por consola o cambiar el string aqui directamente.
54     # Ejemplo de uso desde consola: python augmentation_tuner.py
55     ↪ DataSet/Record_2025-08-08_19-36-02/Images/1754674834877.png
56
57     if len(sys.argv) > 1:
58         target_image = sys.argv[1]
59     else:
60         # Reemplaza esto con una ruta valida real que saques de dataset_train.csv
61         target_image = "DataSet/Record_2025-08-08_19-36-02/Images/1754674834877.png"
62
63     run_tuning_session(target_image)

```

Listing 5: augmentation_tuner.py — stress-test visual del color jitter

`run_tuning_session` toma una imagen y le aplica 15 veces un `ColorJitter` agresivo (brillo $\pm 70\%$, contraste $\pm 60\%$), dibujando una grilla 4×4 . Sirve para chequear a ojo que la augmentation de color no destruye la información útil (la línea blanca sobre el piso gris) mientras simula cambios de iluminación.

Part III

Los motores de modelo

Acá vive la inteligencia. Hay **cuatro motores** (engines), cada uno una forma distinta de plantear el problema. Todos comparten el mismo extractor de imágenes (MobileNetV3-small congelado) y la misma idea de “cuello compartido + cabezas”; lo que cambia es **qué predicen y cómo se entrenan**.

8 nn_perframe.py — el modelo base (per-frame)

Es el modelo fundacional y, como veremos, **el mejor**. Una imagen entra, y salen 4 cosas: la dirección del motor A (3 clases), la del motor B (3 clases), la velocidad de A y la de B (regresión). Todos los demás motores reutilizan piezas de éste.

```
1 """
```

```

2  nn_perframe.py -- MOTOR de entrenamiento PER-FRAME (no es un entrypoint).
3  =====
4  Modelo: IMAGEN (1 frame) -> control por motor (4 salidas):
5      - behaviorA, behaviorB in {STOP, FORWARD, BACKWARD} (clasificacion, 3 clases)
6      - speedA, speedB in [0,100] (regresion, normalizada a
        ↪ [0,1])
7
8  Este archivo NO se corre directo: expone las piezas (Dataset, red, eval, plots) y
9  'run_experiment(cfg, tag)'. Cada modelo concreto vive en su propio 'model_NN*.py',
10 que arma una 'Config' CONGELADA y llama aca. Asi "se ve el modelo en cada codigo"
11 sin duplicar el train-loop. Ver convencion en 'results/README.md'.
12
13     model_01_perframe.py -> run_experiment(CFG, "model_01_perframe")
14     """
15
16 import os
17 import pandas as pd
18 import numpy as np
19 import matplotlib.pyplot as plt
20 import seaborn as sns
21 import torch
22 from torch import nn
23 from torch.utils.data import Dataset, DataLoader
24 import torchvision.transforms as T
25 import torchvision.transforms.functional as TF
26 from torchvision.models import mobilenet_v3_small, MobileNet_V3_Small_Weights
27 import pytorch_lightning as pl
28 from pytorch_lightning.loggers import CSVLogger
29 from pytorch_lightning.callbacks import ModelCheckpoint, LearningRateMonitor
30 from sklearn.metrics import f1_score, confusion_matrix, classification_report
31 from PIL import Image
32
33 from config import active_params_text
34
35 DIR_MAP = {"STOP": 0, "FORWARD": 1, "BACKWARD": 2}
36 DIR_INV = {v: k for k, v in DIR_MAP.items()}
37 SPEED_SCALE = 100.0
38 TURNS = {"LEFT", "RIGHT"}
39
40
41 # =====
42 # 1. DATASET (resampling + flip abstracto, todo desde cfg)
43 # =====
44 class AutonomousDriveDataset(Dataset):
45     def __init__(self, df, is_train=True, cfg=None):
46         self.df = df.reset_index(drop=True)
47         self.is_train = is_train
48         self.cfg = cfg
49
50         self.resize = T.Resize((cfg.image_size, cfg.image_size), antialias=True)
51         self.jitter = T.ColorJitter(cfg.jitter_brightness, cfg.jitter_contrast,
52                                     cfg.jitter_saturation, cfg.jitter_hue)
53         self.normalize = T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
54 ↪ 0.225])
55
56         # En train aplicamos los resample_factor por clase (sobre/sub-muestreo); en
57 ↪ val, no.
58         self.index = self._build_resampled_index() if is_train else
59 ↪ np.arange(len(self.df))

```



```

57
58 def _build_resampled_index(self):
59     rng = np.random.default_rng(self.cfg.seed)
60     factors = self.cfg.resample_factor
61     beh = self.df["behavior"].values
62     idxs = []
63     for i in range(len(self.df)):
64         f = float(factors.get(beh[i], 1.0))
65         n = int(f)
66         idxs.extend([i] * n)
67         frac = f - n
68         if frac > 0 and rng.random() < frac:
69             idxs.append(i)
70     return np.array(idxs, dtype=int)
71
72 def resampled_frames(self):
73     """DataFrame de los frames efectivos (post resample) -- para calcular
↪ class_weights."""
74     return self.df.iloc[self.index]
75
76 def __len__(self):
77     return len(self.index)
78
79 def __getitem__(self, k):
80     row = self.df.iloc[self.index[k]]
81     img = Image.open(row['image_path']).convert('RGB')
82     speedA, speedB = float(row['speedA']), float(row['speedB'])
83     dirA, dirB = row['behaviorA'], row['behaviorB']
84
85     img = self.resize(img)
86
87     if self.is_train:
88         img = self.jitter(img)
89         p = self.cfg.flip_prob_turns if row['behavior'] in TURNS else
↪ self.cfg.flip_prob_straight
90         if torch.rand(1).item() < p:
91             img = TF.hflip(img)
92             speedA, speedB = speedB, speedA    # swap motor A<->B
93             dirA, dirB = dirB, dirA           # FWD/BWD no cambian; el swap basta
94
95     img = self.normalize(TF.to_tensor(img))
96     return img, {
97         "dirA": torch.tensor(DIR_MAP[dirA], dtype=torch.long),
98         "dirB": torch.tensor(DIR_MAP[dirB], dtype=torch.long),
99         "spdA": torch.tensor(speedA / SPEED_SCALE, dtype=torch.float32),
100        "spdB": torch.tensor(speedB / SPEED_SCALE, dtype=torch.float32),
101    }
102
103
104 # =====
105 # 2. MODELO MULTI-CABEZA (arquitectura desde cfg)
106 # =====
107 class MotorControlNet(pl.LightningModule):
108     def __init__(self, neck_hidden=(256,), dropout=0.4, backbone_frozen=True,
109                  lr=1e-3, lambda_speed=1.0, weight_decay=1e-4,
110                  scheduler_step=7, scheduler_gamma=0.1, class_weights=None):
111         super().__init__()
112         self.save_hyperparameters(ignore=['class_weights'])
113

```

```

114     backbone = mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT)
115     feat_dim = backbone.classifier[0].in_features # 576
116     backbone.classifier = nn.Identity()
117     if backbone_frozen:
118         for p in backbone.features.parameters():
119             p.requires_grad = False
120     self.backbone = backbone
121
122     # Cuello compartido: una capa (Linear+Hardswish+Dropout) por entrada de
    ↪ neck_hidden.
123     layers, d = [], feat_dim
124     for h in neck_hidden:
125         layers += [nn.Linear(d, h), nn.Hardswish(), nn.Dropout(dropout)]
126         d = h
127     self.neck = nn.Sequential(*layers)
128     self.head_dirA = nn.Linear(d, 3)
129     self.head_dirB = nn.Linear(d, 3)
130     self.head_spdA = nn.Linear(d, 1)
131     self.head_spdB = nn.Linear(d, 1)
132
133     if class_weights is not None:
134         self.register_buffer("class_weights", class_weights)
135     else:
136         self.class_weights = None
137     self.l1 = nn.L1Loss()
138     self._reset_val_store()
139
140     def _reset_val_store(self):
141         self.val_store = {k: [] for k in ["dirA_p", "dirA_t", "dirB_p", "dirB_t",
142                                           "spdA_p", "spdA_t", "spdB_p", "spdB_t"]}
143
144     def forward(self, x):
145         z = self.neck(self.backbone(x))
146         return (self.head_dirA(z), self.head_dirB(z),
147               torch.sigmoid(self.head_spdA(z)).squeeze(1),
148               torch.sigmoid(self.head_spdB(z)).squeeze(1))
149
150     def _compute_loss(self, out, tgt):
151         logA, logB, spA, spB = out
152         ce = nn.functional.cross_entropy
153         w = self.class_weights
154         loss_dir = ce(logA, tgt["dirA"], weight=w) + ce(logB, tgt["dirB"], weight=w)
155         loss_spd = self.l1(spA, tgt["spdA"]) + self.l1(spB, tgt["spdB"])
156         return loss_dir + self.hparams.lambda_speed * loss_spd, loss_dir, loss_spd
157
158     def training_step(self, batch, batch_idx):
159         x, tgt = batch
160         loss, loss_dir, loss_spd = self._compute_loss(self(x), tgt)
161         self.log('train_loss', loss, on_step=False, on_epoch=True, prog_bar=True)
162         self.log('train_loss_dir', loss_dir, on_step=False, on_epoch=True)
163         self.log('train_loss_spd', loss_spd, on_step=False, on_epoch=True)
164         return loss
165
166     def on_validation_epoch_start(self):
167         self._reset_val_store()
168
169     def validation_step(self, batch, batch_idx):
170         x, tgt = batch
171         out = self(x)

```

```

172     loss, _, _ = self._compute_loss(out, tgt)
173     logA, logB, spA, spB = out
174     self.log('val_loss', loss, on_step=False, on_epoch=True, prog_bar=True)
175     s = self.val_store
176     s["dirA_p"].append(logA.argmax(1).cpu());
↪ s["dirA_t"].append(tgt["dirA"].cpu())
177     s["dirB_p"].append(logB.argmax(1).cpu());
↪ s["dirB_t"].append(tgt["dirB"].cpu())
178     s["spdA_p"].append(spA.float().cpu()); s["spdA_t"].append(tgt["spdA"].cpu())
179     s["spdB_p"].append(spB.float().cpu()); s["spdB_t"].append(tgt["spdB"].cpu())
180
181     def on_validation_epoch_end(self):
182         s = {k: torch.cat(v).numpy() for k, v in self.val_store.items()}
183         f1A = f1_score(s["dirA_t"], s["dirA_p"], average='macro', labels=[0, 1, 2],
↪ zero_division=0)
184         f1B = f1_score(s["dirB_t"], s["dirB_p"], average='macro', labels=[0, 1, 2],
↪ zero_division=0)
185         maeA = np.abs(s["spdA_p"] - s["spdA_t"]).mean() * SPEED_SCALE
186         maeB = np.abs(s["spdB_p"] - s["spdB_t"]).mean() * SPEED_SCALE
187         # recall de full-stop conjunto (ambos motores STOP a la vez) -- metrica de
↪ seguridad
188         true_stop = (s["dirA_t"] == 0) & (s["dirB_t"] == 0)
189         pred_stop = (s["dirA_p"] == 0) & (s["dirB_p"] == 0)
190         fs = float((pred_stop & true_stop).sum() / true_stop.sum()) if
↪ true_stop.sum() else 0.0
191         self.log('val_f1_A', f1A, prog_bar=True)
192         self.log('val_f1_B', f1B, prog_bar=True)
193         self.log('val_dir_f1_macro', (f1A + f1B) / 2, prog_bar=True)
194         self.log('val_mae_A', maeA, prog_bar=True)
195         self.log('val_mae_B', maeB, prog_bar=True)
196         self.log('val_fullstop_recall', fs, prog_bar=True)
197
198     def configure_optimizers(self):
199         opt = torch.optim.AdamW(filter(lambda p: p.requires_grad, self.parameters()),
200                                  lr=self.hparams.lr,
↪ weight_decay=self.hparams.weight_decay)
201         sched = torch.optim.lr_scheduler.StepLR(opt,
↪ step_size=self.hparams.scheduler_step,
202                                                  gamma=self.hparams.scheduler_gamma)
203         return [opt], [sched]
204
205
206     # =====
207     # 3. HARNESS DE INFERENCIA (decoder a pines)
208     # =====
209     def predictions_to_control(dirA, dirB, speedA, speedB, speed_eps=1.0):
210         """Traduce la salida abstracta a pines del L298N (estados siempre validos)."""
211         if speedA < speed_eps:
212             dirA = "STOP"
213         if speedB < speed_eps:
214             dirB = "STOP"
215         gpio = {"FORWARD_A": (1, 0), "BACKWARD_A": (0, 1), "STOP_A": (0, 0),
216               "FORWARD_B": (0, 1), "BACKWARD_B": (1, 0), "STOP_B": (0, 0)}
217         g1, g2 = gpio[f"{dirA}_A"]; g3, g4 = gpio[f"{dirB}_B"]
218         return {"speedA": 0.0 if dirA == "STOP" else round(speedA, 1),
219               "speedB": 0.0 if dirB == "STOP" else round(speedB, 1),
220               "GPIO1": g1, "GPIO2": g2, "GPIO3": g3, "GPIO4": g4}
221
222

```

```

223 # =====
224 # 4. GRAFICADO + EVALUACION LOCAL
225 # =====
226 def plot_training_progress(metrics_csv, out_png="training_progress.png", cfg=None):
227     if not os.path.exists(metrics_csv):
228         print(f"[WARN] No se encontro {metrics_csv}; no se grafica el progreso.")
229         return
230     m =
231     ↪ pd.read_csv(metrics_csv).groupby("epoch").mean(numeric_only=True).reset_index()
232     panels = [
233         ("Loss total", [("train_loss", "Train"), ("val_loss", "Val")]),
234         ↪ ("F1 macro direccion (Val)", [("val_f1_A", "Motor A"), ("val_f1_B", "Motor
235         ↪ B")]),
236         ("MAE velocidad (Val, 0-100)", [("val_mae_A", "Motor A"), ("val_mae_B",
237         ↪ "Motor B")]),
238         ("Learning Rate", [("lr-AdamW", "LR")]),
239     ]
240     fig, axes = plt.subplots(2, 2, figsize=(14, 9))
241     fig.suptitle("Progreso de Entrenamiento (local)", fontsize=15)
242     for ax, (title, series) in zip(axes.ravel(), panels):
243         plotted = False
244         for col, label in series:
245             if col in m.columns and m[col].notna().any():
246                 d = m[["epoch", col]].dropna()
247                 ax.plot(d["epoch"], d[col], marker="o", label=label); plotted = True
248                 ax.set_title(title); ax.set_xlabel("Epoca"); ax.grid(True, alpha=0.3)
249                 ax.legend() if plotted else ax.text(0.5, 0.5, "sin datos", ha="center",
250                 ↪ va="center", transform=ax.transAxes)
251     fig.text(0.5, 0.005, "Parametros: " + active_params_text(cfg).replace("\n", " |
252     ↪ "),
253             ha="center", fontsize=8, family="monospace")
254     plt.tight_layout(rect=[0, 0.03, 1, 1]); plt.savefig(out_png, dpi=150);
255     ↪ plt.close(fig)
256     print(f"\n[graf] Curvas guardadas en '{out_png}'")
257
258 @torch.no_grad()
259 def evaluate_and_plot(model, loader, device, out_png="eval_confusion.png",
260                      report_path=None, cfg=None):
261     model.eval().to(device)
262     pA, tA, pB, tB, eA, eB = [], [], [], [], [], []
263     for x, tgt in loader:
264         logA, logB, spa, spb = model(x.to(device))
265         pA.append(logA.argmax(1).cpu()); tA.append(tgt["dirA"])
266         pB.append(logB.argmax(1).cpu()); tB.append(tgt["dirB"])
267         eA.append((spa.cpu() - tgt["spdA"]).abs()); eB.append((spb.cpu() -
268         ↪ tgt["spdB"]).abs())
269     pA, tA = torch.cat(pA).numpy(), torch.cat(tA).numpy()
270     pB, tB = torch.cat(pB).numpy(), torch.cat(tB).numpy()
271     names = [DIR_INV[i] for i in range(3)]
272     maeA = round(float(torch.cat(eA).mean()) * SPEED_SCALE, 2)
273     maeB = round(float(torch.cat(eB).mean()) * SPEED_SCALE, 2)
274
275     lines = []
276     lines.append("==== MOTOR A (IZQ) ====")
277     lines.append(classification_report(tA, pA, labels=[0, 1, 2], target_names=names,
278     ↪ zero_division=0))
279     lines.append(f"MAE velocidad A: {maeA}")
280     lines.append("\n==== MOTOR B (DER) ====")

```

```

274 lines.append(classification_report(tB, pB, labels=[0, 1, 2], target_names=names,
    ↪ zero_division=0))
275 lines.append(f"MAE velocidad B: {maeB}")
276 report = "\n".join(lines)
277 print("\n" + report)
278 if report_path:
279     with open(report_path, "w") as f:
280         f.write(report + "\n")
281     print(f"[txt] Reporte guardado en '{report_path}'")
282
283 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
284 for ax, (p, t, title) in zip(axes, [(pA, tA, "Motor A (IZQ)"), (pB, tB, "Motor B
    ↪ (DER)"))]:
285     sns.heatmap(confusion_matrix(t, p, labels=[0, 1, 2]), annot=True, fmt='d',
    ↪ cmap='Blues',
286                 xticklabels=names, yticklabels=names, ax=ax)
287     ax.set_title(f"Confusion -- {title}"); ax.set_xlabel("Predicho");
    ↪ ax.set_ylabel("Real")
288 fig.text(0.5, 0.005, "Parametros: " + active_params_text(cfg).replace("\n", " |
    ↪ "),
289         ha="center", fontsize=8, family="monospace")
290 plt.tight_layout(rect=[0, 0.04, 1, 1]); plt.savefig(out_png, dpi=150);
    ↪ plt.close(fig)
291 print(f"[graf] Matrices de confusion guardadas en '{out_png}'")
292
293
294 # =====
295 # 5. UTILIDADES DE EXPERIMENTO
296 # =====
297 def compute_class_weights(frames):
298     pooled = pd.concat([frames["behaviorA"], frames["behaviorB"]]).map(DIR_MAP)
299     counts = pooled.value_counts().reindex([0, 1, 2]).fillna(0)
300     w = counts.sum() / (3 * counts.replace(0, np.nan))
301     return torch.tensor(w.fillna(0).values, dtype=torch.float32)
302
303
304 def save_config_snapshot(cfg, path, extra=""):
305     """Vuelca los hiperparametros CONGELADOS de este experimento a un .txt legible."""
306     from dataclasses import asdict
307     with open(path, "w") as f:
308         if extra:
309             f.write(extra.rstrip() + "\n\n")
310         for k, v in asdict(cfg).items():
311             f.write(f"{k} = {v}\n")
312
313
314 def make_trainer(cfg, out, ckpt_prefix):
315     """Trainer Lightning que detecta GPU/CPU solo (corre en la Jetson o en la
    ↪ laptop)."""
316     use_gpu = torch.cuda.is_available()
317     csv_logger = CSVLogger(save_dir=os.path.join(out, "logs"), name="")
318     checkpoint_callback = ModelCheckpoint(
319         monitor='val_dir_f1_macro', mode='max', save_top_k=1,
320         dirpath=os.path.join(out, "checkpoints"),
321         filename=ckpt_prefix + '-{epoch:02d}-{val_dir_f1_macro:.3f}')
322     lr_monitor = LearningRateMonitor(logging_interval='epoch')
323     trainer = pl.Trainer(
324         max_epochs=cfg.max_epochs, accelerator='auto', devices=1, logger=csv_logger,
325         callbacks=[checkpoint_callback, lr_monitor],

```

```

326         precision='16-mixed' if use_gpu else '32-true')
327     return trainer, csv_logger, checkpoint_callback
328
329
330 def run_experiment(cfg, tag, results_dir="results"):
331     """Entrena el modelo PER-FRAME con la 'cfg' congelada y deja TODO en
    ↪ results/<tag>/."""
332     torch.set_float32_matmul_precision('medium')
333     out = os.path.join(results_dir, tag)
334     os.makedirs(out, exist_ok=True)
335     save_config_snapshot(cfg, os.path.join(out, "config_used.txt"),
336                         extra=f"# {tag} -- modelo PER-FRAME (nn_perframe.py)")
337
338     print(f"=== {tag} | PER-FRAME === Cargando CSVs...")
339     train_df = pd.read_csv(cfg.train_csv)
340     val_df = pd.read_csv(cfg.val_csv)
341
342     train_ds = AutonomousDriveDataset(train_df, is_train=True, cfg=cfg)
343     val_ds = AutonomousDriveDataset(val_df, is_train=False, cfg=cfg)
344     print(f"Frames -> Train: {len(train_df)} (efectivos tras resample:
    ↪ {len(train_ds)}) | Val: {len(val_df)}")
345     print("Parametros activos:\n" + active_params_text(cfg))
346
347     class_weights = compute_class_weights(train_ds.resampled_frames()) if
    ↪ cfg.use_class_weights else None
348     if class_weights is not None:
349         print(f"Pesos de clase {[DIR_INV[i] for i in range(3)]}:
    ↪ {class_weights.tolist()}")
350
351     train_loader = DataLoader(train_ds, batch_size=cfg.batch_size, shuffle=True,
352                             num_workers=cfg.num_workers, pin_memory=True)
353     val_loader = DataLoader(val_ds, batch_size=cfg.batch_size, shuffle=False,
354                           num_workers=cfg.num_workers, pin_memory=True)
355
356     model = MotorControlNet(
357         neck_hidden=tuple(cfg.neck_hidden), dropout=cfg.dropout,
    ↪ backbone_frozen=cfg.backbone_frozen,
358         lr=cfg.lr, lambda_speed=cfg.lambda_speed, weight_decay=cfg.weight_decay,
359         scheduler_step=cfg.scheduler_step, scheduler_gamma=cfg.scheduler_gamma,
360         class_weights=class_weights,
361     )
362
363     trainer, csv_logger, checkpoint_callback = make_trainer(cfg, out,
    ↪ ckpt_prefix="motorctrl")
364     print(f"\n[run] ENTRENANDO {tag} (MotorControlNet) [run]")
365     trainer.fit(model, train_loader, val_loader)
366
367     plot_training_progress(os.path.join(csv_logger.log_dir, "metrics.csv"),
368                           out_png=os.path.join(out, "training_progress.png"),
    ↪ cfg=cfg)
369     best = checkpoint_callback.best_model_path
370     if best and os.path.exists(best):
371         print(f"\nEvaluando mejor checkpoint: {best}")
372         model = MotorControlNet.load_from_checkpoint(best,
    ↪ class_weights=class_weights)
373     device = "cuda" if torch.cuda.is_available() else "cpu"
374     evaluate_and_plot(model, val_loader, device,
375                      out_png=os.path.join(out, "eval_confusion.png"),
376                      report_path=os.path.join(out, "report.txt"), cfg=cfg)

```

377

```
print(f"\n[OK] {tag} listo. Resultados en {out}/")
```

Listing 6: nn_perframe.py — Dataset, red, perdida, metricas, harness y pipeline

1. AutonomousDriveDataset — cómo se sirve cada ejemplo

Hereda de `Dataset` de PyTorch. En `__init__` arma las transformaciones (resize a 224, color jitter, normalización con las estadísticas de ImageNet). La pieza astuta es `_build_resampled_index`: en train, construye un *índice expandido* donde cada frame aparece tantas veces como diga su `resample_factor` (con la parte fraccionaria resuelta al azar). Así el sobre/sub-muestreo por clase se logra sin duplicar imágenes en disco. `__getitem__` abre la imagen, y si es train aplica el *flip abstracto*: con cierta probabilidad espeja la imagen y **intercambia motor A↔B** (velocidad y dirección). Es clave que el flip *no* toca los bits GPIO: como la convención es asimétrica, swappear bits invertiría adelante con atrás; en cambio swappear motores convierte correctamente un giro a la izquierda en uno a la derecha.

2. MotorControlNet — la red (un LightningModule)

- `__init__`: carga MobileNetV3-small con pesos de ImageNet, le saca el clasificador, y **congela** las features (no se entrenan; se aprovecha lo que ya saben de bordes y texturas). Arma el *cuello* (una secuencia de Linear+Hardswish+Dropout por cada valor de `neck_hidden`) y **4 cabezas**: `head_dirA`, `head_dirB` (3 logits cada una) y `head_spdA`, `head_spdB` (1 valor). Registra los `class_weights` si se usan.
- `forward`: pasa la imagen por backbone → cuello → las 4 cabezas. Las velocidades salen por un `sigmoid` (las acota a $[0, 1]$, que luego escalamos a 0–100).
- `_compute_loss`: la pérdida combinada $CE(dirA) + CE(dirB) + \lambda (L1(spda) + L1(spdB))$. La cross-entropy usa los pesos de clase (para que BACKWARD, la clase rara, pese más).
- `training_step` / `validation_step`: el ciclo estándar de Lightning. En validación **no** se calcula el F1 por batch; se *acumulan* todas las predicciones de la época en `val_store`.
- `on_validation_epoch_end`: con todo acumulado, calcula el **F1 macro por motor** (la métrica que de verdad importa, no el accuracy), el **MAE de velocidad**, y el **recall de full-stop conjunto** (que ambos motores estén en STOP a la vez: un evento de seguridad). Calcular el F1 sobre la época entera (y no promediar F1 de batches) es importante para que el número sea correcto.
- `configure_optimizers`: AdamW solo sobre los parámetros entrenables (el backbone congelado queda afuera) + un scheduler de tipo StepLR.

3. predictions_to_control — el harness de inferencia

Esta función traduce la salida *abstracta* del modelo (direcciones + velocidades) a **pines reales del L298N**, con reglas de consistencia: si la velocidad es casi 0 fuerza STOP; si la dirección es STOP pone los GPIO en (0,0) y la velocidad en 0. Gracias a esto el modelo **nunca** puede emitir un estado físicamente inválido (como los 4 GPIO en 1). Mantener esta lógica *fuera* de la red es una decisión de diseño: la red predice algo abstracto y aprendible, y un decodificador determinista garantiza la validez física.

4. Evaluación y graficado

`plot_training_progress` dibuja las curvas (loss, F1 por motor, MAE, learning rate) desde el CSV de métricas. `evaluate_and_plot` corre el mejor checkpoint sobre validación, imprime

y guarda el *classification report* por motor, el MAE, y las **matrices de confusión** 3×3 por motor. `compute_class_weights` calcula los pesos inversos a frecuencia (juntando ambos motores). `run_experiment(cfg, tag)` es el pipeline completo: carga datos, arma el modelo, entrena, y deja *todo* ordenado en `results/<tag>/`. `make_trainer` detecta GPU/CPU solo (corre igual en la Jetson o en la laptop).

9 nn_sequence.py — el modelo temporal (CNN + GRU/LSTM)

La hipótesis: como la *velocidad* no se ve en un solo frame (un auto quieto y uno avanzando despacio en el mismo lugar son idénticos), darle al modelo una **secuencia** de frames debería ayudar. Este motor extrae features por frame con el MobileNet congelado, las pasa por una red recurrente (GRU o LSTM), y predice la acción del **último** frame de la ventana (esquema “many-to-one”, causal y desplegable).

```

1  """
2  nn_sequence.py -- MOTOR de entrenamiento SECUENCIAL CNN + GRU/LSTM (no es un
    ↪ entrypoint).
3  =====
4  Modelo: SECUENCIA de imagenes -> control por motor (mismas 4 salidas que nn_perframe).
5
6  POR QUE una recurrente: el error dominante del per-frame es STOP vs FORWARD, porque la
7  VELOCIDAD no esta en un unico frame (un auto quieto y uno avanzando despacio en el
    ↪ mismo
8  punto son identicos). Una secuencia muestra si la escena avanza -> la GRU/LSTM
    ↪ resuelve
9  esa ambigüedad temporal.
10
11 ARQUITECTURA: MobileNetV3-small CONGELADO como extractor por-frame (TimeDistributed)
12 -> GRU/LSTM sobre los T vectores -> cuello compartido -> 4 cabezas. Se reutiliza toda
    ↪ la
13 logica de loss/metricas/eval/plots de 'nn_perframe' (herencia) para que la comparacion
14 con el modelo por-frame sea justa.
15
16 Este archivo NO se corre directo: expone 'run_experiment(cfg, tag)'. Cada variante RNN
17 vive en su 'model_NN_*.py' con su 'Config' congelada (ver 'results/README.md'):
18     model_02_gru_v1.py model_03_gru_v2.py model_04_lstm.py
    ↪ model_05_gru_v3_antiredun.py
19 """
20
21 import os
22 import random
23 import warnings
24
25 # El ajuste de hue (PIL) castea floats negativos a uint8; numpy>=1.24 lo avisa como
26 # "invalid value encountered in cast". Es BENIGNO: el wraparound es justo lo que el
    ↪ hue
27 # circular necesita. Lo silenciamos para no inundar el log (no cambia el resultado).
28 warnings.filterwarnings("ignore", message="invalid value encountered in cast")
29
30 import numpy as np
31 import pandas as pd
32 import torch
33 from torch import nn
34 from torch.utils.data import Dataset, DataLoader
35 import torchvision.transforms as T
36 import torchvision.transforms.functional as TF
37 from torchvision.models import mobilenet_v3_small, MobileNet_V3_Small_Weights

```

```

38 import pytorch_lightning as pl
39 from PIL import Image
40
41 from config import active_params_text
42 # Reutilizamos del modelo por-frame: mapeos, loss/metricas (herencia), eval, plots,
43   ↪ utils.
44 from nn_perframe import (
45     MotorControlNet, DIR_MAP, DIR_INV, SPEED_SCALE, TURNS,
46     compute_class_weights, evaluate_and_plot, plot_training_progress,
47     save_config_snapshot, make_trainer,
48 )
49
50 # =====
51 # 1. DATASET DE SECUENCIAS (ventana many-to-one, todo desde cfg)
52 # =====
53 class SequenceDriveDataset(Dataset):
54     def __init__(self, df, is_train=True, cfg=None):
55         self.df = df.reset_index(drop=True)
56         self.is_train = is_train
57         self.cfg = cfg
58
59         self.resize = T.Resize((cfg.image_size, cfg.image_size), antialias=True)
60         self.normalize = T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
61   ↪ 0.225])
62
63         # Ventanas validas: una por cada frame que tenga (seq_len-1)*stride frames
64   ↪ previos
65         # DENTRO de su mismo record. En train se expanden con resample_factor por
66   ↪ clase.
67         self.samples = self._build_samples()
68
69     def _build_samples(self):
70         T_, k = self.cfg.seq_len, self.cfg.seq_stride
71         # Anti-redundancia: en TRAIN subsamplea las ventanas (1 de cada wstep) para
72   ↪ cortar
73         # el solape entre secuencias casi identicas (sliding window a 22fps). Val sin
74   ↪ tocar.
75         wstep = getattr(self.cfg, "seq_window_step", 1) if self.is_train else 1
76         span = (T_ - 1) * k
77         base = []
78         for _, g in self.df.groupby("record", sort=False):
79             idxs = g.sort_values("time_in_ms").index.to_numpy()
80             for pos in range(span, len(idxs), wstep):
81                 win = idxs[pos - span: pos + 1: k] # T_ indices, orden temporal
82   ↪ ascendente
83                 if len(win) == T_:
84                     base.append(win)
85             if not self.is_train:
86                 return base
87             # Resample por el behavior global del frame-ETIQUETA (el ultimo de la
88   ↪ ventana).
89             rng = np.random.default_rng(self.cfg.seed)
90             factors = self.cfg.resample_factor
91             beh = self.df["behavior"].values
92             out = []
93             for win in base:
94                 f = float(factors.get(beh[win[-1]], 1.0))
95                 n = int(f)

```

```

89         out.extend([win] * n)
90         frac = f - n
91         if frac > 0 and rng.random() < frac:
92             out.append(win)
93     return out
94
95     def label_frames(self):
96         """DataFrame de los frames-etiqueta (ultimo de cada ventana, post-resample).
97         Sirve para calcular class_weights sobre lo que el modelo realmente ve."""
98         last = [w[-1] for w in self.samples]
99         return self.df.iloc[last]
100
101     def __len__(self):
102         return len(self.samples)
103
104     # --- color jitter CONSISTENTE en toda la secuencia (mismos factores por frame)
105     ↪ ---
106     def _sample_jitter(self):
107         c = self.cfg
108         b = random.uniform(max(0.0, 1 - c.jitter_brightness), 1 + c.jitter_brightness)
109         co = random.uniform(max(0.0, 1 - c.jitter_contrast), 1 + c.jitter_contrast)
110         s = random.uniform(max(0.0, 1 - c.jitter_saturation), 1 + c.jitter_saturation)
111         h = random.uniform(-c.jitter_hue, c.jitter_hue)
112         return b, co, s, h
113
114     @staticmethod
115     def _apply_jitter(img, jit):
116         b, co, s, h = jit
117         img = TF.adjust_brightness(img, b)
118         img = TF.adjust_contrast(img, co)
119         img = TF.adjust_saturation(img, s)
120         img = TF.adjust_hue(img, h)
121         return img
122
123     def __getitem__(self, k):
124         win = self.samples[k]
125         rows = self.df.iloc[win]
126         label = rows.iloc[-1]
127         speedA, speedB = float(label["speedA"]), float(label["speedB"])
128         dirA, dirB = label["behaviorA"], label["behaviorB"]
129
130         do_flip, jit = False, None
131         if self.is_train:
132             # El flip se decide UNA vez y se aplica IGUAL a toda la secuencia (+ swap
133             ↪ A<->B).
134             p = self.cfg.flip_prob_turns if label["behavior"] in TURNS else
135             ↪ self.cfg.flip_prob_straight
136             do_flip = torch.rand(1).item() < p
137             if do_flip:
138                 speedA, speedB = speedB, speedA
139                 dirA, dirB = dirB, dirA
140                 jit = self._sample_jitter()
141
142         frames = []
143         for path in rows["image_path"]:
144             img = self.resize(Image.open(path).convert("RGB"))
145             if self.is_train:
146                 img = self._apply_jitter(img, jit)
147             if do_flip:

```

```

145         img = TF.hflip(img)
146         frames.append(self.normalize(TF.to_tensor(img)))
147     seq = torch.stack(frames)  # (T, C, H, W)
148
149     return seq, {
150         "dirA": torch.tensor(DIR_MAP[dirA], dtype=torch.long),
151         "dirB": torch.tensor(DIR_MAP[dirB], dtype=torch.long),
152         "spdA": torch.tensor(speedA / SPEED_SCALE, dtype=torch.float32),
153         "spdB": torch.tensor(speedB / SPEED_SCALE, dtype=torch.float32),
154     }
155
156
157 # =====
158 # 2. MODELO CNN + GRU/LSTM (hereda loss/metricas de MotorControlNet)
159 # =====
160 class MotorControlGRU(MotorControlNet):
161     """Backbone congelado por-frame -> GRU/LSTM -> cuello -> 4 cabezas.
162     Reutiliza _compute_loss / training_step / validation_step / on_validation_epoch_*
163     / configure_optimizers de MotorControlNet (misma loss y misma metrica F1)."""
164
165     def __init__(self, neck_hidden=(256,), dropout=0.4, rnn_type="GRU",
166         ↪ rnn_hidden=128,
167         rnn_layers=1, lr=1e-3, lambda_speed=1.0, weight_decay=1e-4,
168         scheduler_step=7, scheduler_gamma=0.1, class_weights=None):
169         ↪ Lightning.
170         pl.LightningModule.__init__(self)
171         ↪ Referenciar __class__ fuerza la creacion de la celda __class__ del metodo:
172         ↪ sin
173         ↪ ella, save_hyperparameters de Lightning 2.x no reconoce este __init__ como
174         ↪ el de
175         ↪ una clase y NO captura ningun hparam (quedaria hparams.lr vacio ->
176         ↪ KeyError).
177         _ = __class__  # noqa: F821
178         self.save_hyperparameters("neck_hidden", "dropout", "rnn_type", "rnn_hidden",
179             "rnn_layers", "lr", "lambda_speed", "weight_decay",
180             "scheduler_step", "scheduler_gamma")
181
182         backbone = mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT)
183         feat_dim = backbone.classifier[0].in_features  # 576
184         backbone.classifier = nn.Identity()
185         for p in backbone.parameters():
186             p.requires_grad = False
187         backbone.eval()  # extractor fijo: BN usa stats de ImageNet, no se actualiza
188         self.backbone = backbone
189
190         rnn_cls = nn.LSTM if rnn_type.upper() == "LSTM" else nn.GRU
191         self.rnn = rnn_cls(input_size=feat_dim, hidden_size=rnn_hidden,
192             ↪ num_layers=rnn_layers,
193             batch_first=True, dropout=(dropout if rnn_layers > 1 else
194             ↪ 0.0))
195
196         ↪ Cuello compartido sobre el ultimo estado oculto (igual idea que el
197         ↪ por-frame).
198         layers, d = [], rnn_hidden
199         for h in neck_hidden:
200             layers += [nn.Linear(d, h), nn.Hardswish(), nn.Dropout(dropout)]
201             d = h
202         self.neck = nn.Sequential(*layers)

```

```

196     self.head_dirA = nn.Linear(d, 3)
197     self.head_dirB = nn.Linear(d, 3)
198     self.head_spdA = nn.Linear(d, 1)
199     self.head_spdB = nn.Linear(d, 1)
200
201     if class_weights is not None:
202         self.register_buffer("class_weights", class_weights)
203     else:
204         self.class_weights = None
205     self.l1 = nn.L1Loss()
206     self._reset_val_store()
207
208     def forward(self, x): # x: (B, T, C, H, W)
209         B, Tlen = x.shape[0], x.shape[1]
210         # Backbone congelado: sin grad y en eval (lo re-aseguramos porque
211         ↪ trainer.train()
212         # lo pondria en modo train cada epoca). Solo entrenan rnn + neck + cabezas.
213         self.backbone.eval()
214         with torch.no_grad():
215             feats = self.backbone(x.reshape(B * Tlen, *x.shape[2:]))
216             feats = feats.view(B, Tlen, -1)
217             out = self.rnn(feats)[0] # GRU y LSTM devuelven (output, hidden);
218         ↪ tomamos output
219         z = self.neck(out[:, -1]) # ultimo timestep -> many-to-one
220         return (self.head_dirA(z), self.head_dirB(z),
221                 torch.sigmoid(self.head_spdA(z)).squeeze(1),
222                 torch.sigmoid(self.head_spdB(z)).squeeze(1))
223
224 # =====
225 # 3. PIPELINE
226 # =====
227 def run_experiment(cfg, tag, results_dir="results"):
228     """Entrena el modelo SECUENCIAL con la 'cfg' congelada y deja TODO en
229     ↪ results/<tag>/."""
230     torch.set_float32_matmul_precision('medium')
231     out = os.path.join(results_dir, tag)
232     os.makedirs(out, exist_ok=True)
233     save_config_snapshot(cfg, os.path.join(out, "config_used.txt"),
234                         extra=f"# {tag} -- modelo SECUENCIAL {cfg.rnn_type}"
235     ↪ (nn_sequence.py))
236
237     print(f"=== {tag} | SECUENCIAL {cfg.rnn_type} === Cargando CSVs...")
238     train_df = pd.read_csv(cfg.train_csv)
239     val_df = pd.read_csv(cfg.val_csv)
240
241     train_ds = SequenceDriveDataset(train_df, is_train=True, cfg=cfg)
242     val_ds = SequenceDriveDataset(val_df, is_train=False, cfg=cfg)
243     print(f"Secuencias (T={cfg.seq_len}, stride={cfg.seq_stride},
244     ↪ wstep={cfg.seq_window_step}) -> "
245           f"Train: {len(train_ds)} | Val: {len(val_ds)}")
246     if len(val_ds) < 50:
247         print(f"[ADVERTENCIA] Solo {len(val_ds)} secuencias de validacion: "
248               ↪ f"la metrica va a ser RUIDOSA (val es corto y T/stride lo reducen
249               ↪ mas).")
250     print("Parametros activos:\n" + active_params_text(cfg))
251
252     class_weights = compute_class_weights(train_ds.label_frames()) if
253     ↪ cfg.use_class_weights else None

```

```

248     if class_weights is not None:
249         print(f"Pesos de clase {[DIR_INV[i] for i in range(3)]}:
↪ {class_weights.tolist()}")
250
251     train_loader = DataLoader(train_ds, batch_size=cfg.batch_size, shuffle=True,
252                               num_workers=cfg.num_workers, pin_memory=True)
253     val_loader = DataLoader(val_ds, batch_size=cfg.batch_size, shuffle=False,
254                             num_workers=cfg.num_workers, pin_memory=True)
255
256     model = MotorControlGRU(
257         neck_hidden=tuple(cfg.neck_hidden), dropout=cfg.dropout,
258         rnn_type=cfg.rnn_type, rnn_hidden=cfg.rnn_hidden, rnn_layers=cfg.rnn_layers,
259         lr=cfg.lr, lambda_speed=cfg.lambda_speed, weight_decay=cfg.weight_decay,
260         scheduler_step=cfg.scheduler_step, scheduler_gamma=cfg.scheduler_gamma,
261         class_weights=class_weights,
262     )
263
264     trainer, csv_logger, checkpoint_callback = make_trainer(cfg, out,
↪ ckpt_prefix="motorgru")
265     print(f"\n[run] ENTRENANDO {tag} (MotorControlGRU / {cfg.rnn_type}) [run]")
266     trainer.fit(model, train_loader, val_loader)
267
268     plot_training_progress(os.path.join(csv_logger.log_dir, "metrics.csv"),
269                             out_png=os.path.join(out, "training_progress.png"),
↪ cfg=cfg)
270     best = checkpoint_callback.best_model_path
271     if best and os.path.exists(best):
272         print(f"\nEvaluando mejor checkpoint: {best}")
273         model = MotorControlGRU.load_from_checkpoint(best,
↪ class_weights=class_weights)
274         device = "cuda" if torch.cuda.is_available() else "cpu"
275         evaluate_and_plot(model, val_loader, device,
276                             out_png=os.path.join(out, "eval_confusion.png"),
277                             report_path=os.path.join(out, "report.txt"), cfg=cfg)
278     print(f"\n[OK] {tag} listo. Resultados en {out}/")

```

Listing 7: nn_sequence.py — Dataset de secuencias y red recurrente

Lo que cambia respecto al per-frame

- **SequenceDriveDataset**: arma *ventanas* de `seq_len` frames con paso `seq_stride`, **dentro** de cada `record/_seqN` (nunca cruza una frontera de sesión). El knob `seq_window_step` es “anti-redundancia”: en train se queda con 1 de cada `N` ventanas, para cortar el enorme solape entre secuencias casi idénticas (consecuencia de los 22 fps). El *color jitter* y el *flip* se deciden **una vez por secuencia** y se aplican igual a todos sus frames (coherencia temporal).
- **MotorControlGRU**: **hereda** de `MotorControlNet` para reutilizar toda la pérdida, métricas y evaluación (la comparación con el per-frame es así justa). Reemplaza el cuello por una `nn.GRU` o `nn.LSTM` sobre los T vectores de features, y toma el último estado oculto. El detalle fino `_ = __class__` fuerza la creación de la celda `__class__` del método, necesaria para que `save_hyperparameters` de Lightning 2.x capture los hiperparámetros.
- **forward**: re-aplica `backbone.eval()` y `torch.no_grad()` sobre el backbone en cada paso (porque el `trainer` lo pondría en modo train cada época); solo entrenan la recurrente, el cuello y las cabezas.

10 nn_signed.py — velocidad con signo (el comando físico)

Acá el target se unifica: **una sola salida continua por motor**, el PWM con signo. La idea es elegante (el modelo predice exactamente lo que va a la rueda) y nos da una métrica nativa: el MAE del PWM con signo.

```

1  """
2  nn_signed.py -- MOTOR de entrenamiento PER-FRAME con VELOCIDAD CON SIGNO (no es
   ↪ entrypoint).
3  =====
4  Modelo: IMAGEN -> 2 variables CONTINUAS (signed_speedA, signed_speedB) in [-100,
   ↪ 100]:
5      > 0 FORWARD    < 0 BACKWARD    = 0 STOP    (la direccion es el SIGNO de la salida)
6
7  Diferencia con nn_perframe: NO hay cabezas de clasificacion de direccion. Una sola
   ↪ salida
8  continua por motor (tanh*100) unifica direccion + velocidad en el comando fisico
   ↪ (PWM).
9
10 PARA QUE BACKWARD SOBREVIVA (sin class-weights, porque no hay clases): la L1 se
   ↪ PONDERA
11 POR MUESTRA segun la direccion REAL de cada motor (peso inverso a la frecuencia, igual
12 idea que las class-weights del modelo por-frame, pero aplicado al termino L1 de cada
   ↪ motor).
13
14 METRICAS (val, calculadas sobre la salida continua):
15     - val_signed_mae_A/B/avg : MAE del PWM con signo (la metrica nativa; titular).
16     - val_dir_f1_A/B/macro   : F1 macro decodificando sign(salida) -> {STOP,FWD,BWD},
17                               para poder COMPARAR con los modelos 00-05 del scoreboard.
18     - val_fullstop_recall    : recall del escenario "ambos motores STOP" (seguridad).
19
20 Este archivo NO se corre directo: expone 'run_experiment(cfg, tag)'. El modelo
   ↪ concreto
21 vive en model_06_signed.py con su Config congelada (lee dataset_*_signed.csv).
22 """
23
24 import os
25 import numpy as np
26 import pandas as pd
27 import matplotlib.pyplot as plt
28 import seaborn as sns
29 import torch
30 from torch import nn
31 from torch.utils.data import Dataset, DataLoader
32 import torchvision.transforms as T
33 import torchvision.transforms.functional as TF
34 from torchvision.models import mobilenet_v3_small, MobileNet_V3_Small_Weights
35 import pytorch_lightning as pl
36 from sklearn.metrics import f1_score, confusion_matrix, classification_report
37 from PIL import Image
38
39 from config import active_params_text
40 from nn_perframe import (
41     DIR_MAP, DIR_INV, SPEED_SCALE, TURNS, compute_class_weights,
42     save_config_snapshot, make_trainer,
43 )
44
45 DIR_EPS_PWM = 1.0 # /signed/ < 1 PWM -> STOP (mismo umbral que el harness de
   ↪ inferencia)

```



```

46
47
48 def signed_to_dir(v_norm, eps_norm):
49     """Decodifica un PWM-con-signo NORMALIZADO (/100) a direccion
50     ↪ {0:STOP,1:FWD,2:BWD}."""
51     out = np.zeros_like(v_norm, dtype=int) # STOP
52     out[v_norm > eps_norm] = DIR_MAP["FORWARD"] # 1
53     out[v_norm < -eps_norm] = DIR_MAP["BACKWARD"] # 2
54     return out
55
56 # =====
57 # 1. DATASET (target = signed speed; flip = swap A<->B con signo intacto)
58 # =====
59 class SignedDriveDataset(Dataset):
60     def __init__(self, df, is_train=True, cfg=None):
61         self.df = df.reset_index(drop=True)
62         self.is_train = is_train
63         self.cfg = cfg
64         self.resize = T.Resize((cfg.image_size, cfg.image_size), antialias=True)
65         self.jitter = T.ColorJitter(cfg.jitter_brightness, cfg.jitter_contrast,
66                                     cfg.jitter_saturation, cfg.jitter_hue)
67         self.normalize = T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
68     ↪ 0.225])
69         self.index = self._build_resampled_index() if is_train else
70         ↪ np.arange(len(self.df))
71
72     def _build_resampled_index(self):
73         rng = np.random.default_rng(self.cfg.seed)
74         factors = self.cfg.resample_factor
75         beh = self.df["behavior"].values
76         idxs = []
77         for i in range(len(self.df)):
78             f = float(factors.get(beh[i], 1.0))
79             n = int(f)
80             idxs.extend([i] * n)
81             frac = f - n
82             if frac > 0 and rng.random() < frac:
83                 idxs.append(i)
84         return np.array(idxs, dtype=int)
85
86     def resampled_frames(self):
87         return self.df.iloc[self.index]
88
89     def __len__(self):
90         return len(self.index)
91
92     def __getitem__(self, k):
93         row = self.df.iloc[self.index[k]]
94         img = Image.open(row['image_path']).convert('RGB')
95         sA, sB = float(row['signed_speedA']), float(row['signed_speedB'])
96         dirA, dirB = row['behaviorA'], row['behaviorB']
97
98         img = self.resize(img)
99         if self.is_train:
100             img = self.jitter(img)
101             p = self.cfg.flip_prob_turns if row['behavior'] in TURNS else
102             ↪ self.cfg.flip_prob_straight
103             if torch.rand(1).item() < p:

```

```

101         img = TF.hflip(img)
102         sA, sB = sB, sA          # swap motor A<->B; el SIGNO no cambia (fwd/bwd
    ↪ es el eje)
103         dirA, dirB = dirB, dirA
104
105         img = self.normalize(TF.to_tensor(img))
106         return img, {
107             "spdA": torch.tensor(sA / SPEED_SCALE, dtype=torch.float32),
108             "spdB": torch.tensor(sB / SPEED_SCALE, dtype=torch.float32),
109             "dirA": torch.tensor(DIR_MAP[dirA], dtype=torch.long), # solo para peso
    ↪ + metrica
110             "dirB": torch.tensor(DIR_MAP[dirB], dtype=torch.long),
111         }
112
113
114 # =====
115 # 2. MODELO (MobileNet congelado -> cuello -> 2 salidas tanh)
116 # =====
117 class SignedControlNet(pl.LightningModule):
118     def __init__(self, neck_hidden=(256,), dropout=0.4, backbone_frozen=True,
119                 lr=1e-3, weight_decay=1e-4, scheduler_step=7, scheduler_gamma=0.1,
120                 dir_weights=None, eps_norm=DIR_EPS_PWM / SPEED_SCALE):
121         super().__init__()
122         self.save_hyperparameters(ignore=['dir_weights'])
123
124         backbone = mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT)
125         feat_dim = backbone.classifier[0].in_features
126         backbone.classifier = nn.Identity()
127         if backbone_frozen:
128             for p in backbone.features.parameters():
129                 p.requires_grad = False
130         self.backbone = backbone
131
132         layers, d = [], feat_dim
133         for h in neck_hidden:
134             layers += [nn.Linear(d, h), nn.Hardswish(), nn.Dropout(dropout)]
135             d = h
136         self.neck = nn.Sequential(*layers)
137         self.head_spdA = nn.Linear(d, 1)
138         self.head_spdB = nn.Linear(d, 1)
139
140         # Pesos por direccion (STOP/FWD/BWD) para ponderar la L1 por muestra.
141         if dir_weights is None:
142             dir_weights = torch.ones(3)
143         self.register_buffer("dir_weights", dir_weights)
144         self._reset_val_store()
145
146     def _reset_val_store(self):
147         self.val_store = {k: [] for k in ["spdA_p", "spdA_t", "spdB_p", "spdB_t",
148                                           "dirA_t", "dirB_t"]}
149
150     def forward(self, x):
151         z = self.neck(self.backbone(x))
152         # tanh acota la salida a [-1,1] -> *no* multiplicamos aca; el target esta
    ↪ /100 en [-1,1].
153         return torch.tanh(self.head_spdA(z)).squeeze(1),
    ↪ torch.tanh(self.head_spdB(z)).squeeze(1)
154
155     def _weighted_l1(self, out, tgt):

```

```

156     pA, pB = out
157     wA = self.dir_weights[tgt["dirA"]] # peso por la direccion REAL de cada
158     ↪ motor
159     wB = self.dir_weights[tgt["dirB"]]
160     loss = (wA * (pA - tgt["spdA"]).abs()).mean() + (wB * (pB -
161     ↪ tgt["spdB"]).abs()).mean()
162     return loss
163
164 def training_step(self, batch, batch_idx):
165     x, tgt = batch
166     loss = self._weighted_l1(self(x), tgt)
167     self.log('train_loss', loss, on_step=False, on_epoch=True, prog_bar=True)
168     return loss
169
170 def on_validation_epoch_start(self):
171     self._reset_val_store()
172
173 def validation_step(self, batch, batch_idx):
174     x, tgt = batch
175     pA, pB = self(x)
176     self.log('val_loss', self._weighted_l1((pA, pB), tgt), on_step=False,
177     ↪ on_epoch=True, prog_bar=True)
178     s = self.val_store
179     s["spdA_p"].append(pA.float().cpu()); s["spdA_t"].append(tgt["spdA"].cpu())
180     s["spdB_p"].append(pB.float().cpu()); s["spdB_t"].append(tgt["spdB"].cpu())
181     s["dirA_t"].append(tgt["dirA"].cpu()); s["dirB_t"].append(tgt["dirB"].cpu())
182
183 def on_validation_epoch_end(self):
184     s = {k: torch.cat(v).numpy() for k, v in self.val_store.items()}
185     eps = self.hparams.eps_norm
186     maeA = np.abs(s["spdA_p"] - s["spdA_t"]).mean() * SPEED_SCALE
187     maeB = np.abs(s["spdB_p"] - s["spdB_t"]).mean() * SPEED_SCALE
188     # decodificar direccion desde el signo para F1 comparable con los modelos
189     ↪ 00-05
190     pdA, pdB = signed_to_dir(s["spdA_p"], eps), signed_to_dir(s["spdB_p"], eps)
191     tdA, tdB = s["dirA_t"], s["dirB_t"]
192     f1A = f1_score(tdA, pdA, average='macro', labels=[0, 1, 2], zero_division=0)
193     f1B = f1_score(tdB, pdB, average='macro', labels=[0, 1, 2], zero_division=0)
194     # recall de full-stop conjunto (ambos motores STOP)
195     true_stop = (tdA == 0) & (tdB == 0)
196     pred_stop = (pdA == 0) & (pdB == 0)
197     fs_recall = float((pred_stop & true_stop).sum() / true_stop.sum()) if
198     ↪ true_stop.sum() else 0.0
199
200     self.log('val_signed_mae_A', maeA, prog_bar=True)
201     self.log('val_signed_mae_B', maeB, prog_bar=True)
202     self.log('val_signed_mae_avg', (maeA + maeB) / 2, prog_bar=True)
203     self.log('val_dir_f1_A', f1A)
204     self.log('val_dir_f1_B', f1B)
205     self.log('val_dir_f1_macro', (f1A + f1B) / 2, prog_bar=True)
206     self.log('val_fullstop_recall', fs_recall, prog_bar=True)
207
208 def configure_optimizers(self):
209     opt = torch.optim.AdamW(filter(lambda p: p.requires_grad, self.parameters()),
210     ↪ lr=self.hparams.lr,
211     ↪ weight_decay=self.hparams.weight_decay)
212     sched = torch.optim.lr_scheduler.StepLR(opt,
213     ↪ step_size=self.hparams.scheduler_step,
214     ↪ gamma=self.hparams.scheduler_gamma)

```

```

208         return [opt], [sched]
209
210
211 # =====
212 # 3. GRAFICADO + EVALUACION
213 # =====
214 def plot_training_progress(metrics_csv, out_png, cfg=None):
215     if not os.path.exists(metrics_csv):
216         print(f"[WARN] No se encontro {metrics_csv}.")
217         return
218     m =
219     ↪ pd.read_csv(metrics_csv).groupby("epoch").mean(numeric_only=True).reset_index()
220     panels = [
221         ("Loss (L1 ponderada)", [("train_loss", "Train"), ("val_loss", "Val")]),
222         ↪ ("MAE PWM con signo (Val)", [("val_signed_mae_A", "Motor A"),
223         ↪ ("val_signed_mae_B", "Motor B")]),
224         ("F1 macro direccion decodificada (Val)", [("val_dir_f1_A", "Motor A"),
225         ↪ ("val_dir_f1_B", "Motor B")]),
226         ("Recall full-stop (Val)", [("val_fullstop_recall", "full-stop")]),
227     ]
228     fig, axes = plt.subplots(2, 2, figsize=(14, 9))
229     fig.suptitle("Progreso -- SignedControlNet (PWM con signo)", fontsize=15)
230     for ax, (title, series) in zip(axes.ravel(), panels):
231         plotted = False
232         for col, label in series:
233             if col in m.columns and m[col].notna().any():
234                 d = m[["epoch", col]].dropna()
235                 ax.plot(d["epoch"], d[col], marker="o", label=label); plotted = True
236                 ax.set_title(title); ax.set_xlabel("Epoca"); ax.grid(True, alpha=0.3)
237                 ax.legend() if plotted else ax.text(0.5, 0.5, "sin datos", ha="center",
238                 ↪ va="center", transform=ax.transAxes)
239         fig.text(0.5, 0.005, "Parametros: " + active_params_text(cfg).replace("\n", " |
240         ↪ "),
241                 ha="center", fontsize=8, family="monospace")
242     plt.tight_layout(rect=[0, 0.03, 1, 1]); plt.savefig(out_png, dpi=150);
243     ↪ plt.close(fig)
244     print(f"\n[graf] Curvas guardadas en '{out_png}'")
245
246
247 @torch.no_grad()
248 def evaluate_and_plot(model, loader, device, out_png, report_path=None, cfg=None):
249     model.eval().to(device)
250     eps = model.hparams.eps_norm
251     pA, tA, pB, tB, eA, eB = [], [], [], [], [], []
252     tdA, tdB = [], []
253     for x, tgt in loader:
254         a, b = model(x.to(device))
255         pA.append(a.cpu()); tA.append(tgt["spdA"]); pB.append(b.cpu());
256         ↪ tB.append(tgt["spdB"]);
257         eA.append((a.cpu() - tgt["spdA"]).abs()); eB.append((b.cpu() -
258         ↪ tgt["spdB"]).abs())
259         tdA.append(tgt["dirA"]); tdB.append(tgt["dirB"])
260     pA, tA_ = torch.cat(pA).numpy(), torch.cat(tA).numpy()
261     pB, tB_ = torch.cat(pB).numpy(), torch.cat(tB).numpy()
262     tdA, tdB = torch.cat(tdA).numpy(), torch.cat(tdB).numpy()
263     pdA, pdB = signed_to_dir(pA, eps), signed_to_dir(pB, eps)
264     maeA = round(float(torch.cat(eA).mean()) * SPEED_SCALE, 2)
265     maeB = round(float(torch.cat(eB).mean()) * SPEED_SCALE, 2)
266     names = [DIR_INV[i] for i in range(3)]

```

```

259 true_stop = (tdA == 0) & (tdB == 0)
260 pred_stop = (pdA == 0) & (pdB == 0)
261 fs = float((pred_stop & true_stop).sum() / true_stop.sum()) if true_stop.sum()
    ↪ else 0.0
263
264 lines = []
265 lines.append("==== MOTOR A (IZQ) -- direccion decodificada de sign(PWM) =====")
266 lines.append(classification_report(tdA, pdA, labels=[0, 1, 2],
    ↪ target_names=names, zero_division=0))
267 lines.append(f"MAE PWM con signo A: {maeA}")
268 lines.append("\n==== MOTOR B (DER) -- direccion decodificada de sign(PWM) =====")
269 lines.append(classification_report(tdB, pdB, labels=[0, 1, 2],
    ↪ target_names=names, zero_division=0))
270 lines.append(f"MAE PWM con signo B: {maeB}")
271 lines.append(f"\nRecall full-stop conjunto (ambos motores STOP): {fs:.3f}")
272 lines.append(f"MAE PWM con signo PROMEDIO (titular): {round((maeA + maeB) / 2,
    ↪ 2)}")
273 report = "\n".join(lines)
274 print("\n" + report)
275 if report_path:
276     with open(report_path, "w") as f:
277         f.write(report + "\n")
278     print(f"[txt] Reporte guardado en '{report_path}'")
279
280 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
281 for ax, (p, t, title) in zip(axes, [(pdA, tdA, "Motor A (IZQ)", (pdB, tdB,
    ↪ "Motor B (DER)"))]):
282     sns.heatmap(confusion_matrix(t, p, labels=[0, 1, 2]), annot=True, fmt='d',
    ↪ cmap='Blues',
283                  xticklabels=names, yticklabels=names, ax=ax)
284     ax.set_title(f"Confusion (dir decodificada) -- {title}");
    ↪ ax.set_xlabel("Predicho"); ax.set_ylabel("Real")
285 fig.suptitle("model_06_signed -- direccion = signo del PWM predicho")
286 plt.tight_layout(rect=[0, 0.04, 1, 1]); plt.savefig(out_png, dpi=150);
    ↪ plt.close(fig)
287 print(f"[graf] Matrices de confusion guardadas en '{out_png}'")
288
289
290 # =====
291 # 4. PIPELINE
292 # =====
293 def run_experiment(cfg, tag, results_dir="results", select_by="mae"):
294     """Entrena el modelo SIGNED con la 'cfg' congelada y deja TODO en results/<tag>/.
295
296     select_by: criterio para elegir el MEJOR checkpoint:
297     "mae" -> menor val_signed_mae_avg (metrica fisica nativa).
298     "f1" -> mayor val_dir_f1_macro (direccion decodificada; comparable con 00-05).
299     """
300     torch.set_float32_matmul_precision('medium')
301     out = os.path.join(results_dir, tag)
302     os.makedirs(out, exist_ok=True)
303     save_config_snapshot(cfg, os.path.join(out, "config_used.txt"),
304                          extra=f"# {tag} -- PER-FRAME, velocidad CON SIGNO
    ↪ (nn_signed.py)")
305
306     print(f"=== {tag} | SIGNED === Cargando CSVs...")
307     train_df = pd.read_csv(cfg.train_csv)
308     val_df = pd.read_csv(cfg.val_csv)

```

```

309 train_ds = SignedDriveDataset(train_df, is_train=True, cfg=cfg)
310 val_ds = SignedDriveDataset(val_df, is_train=False, cfg=cfg)
311 print(f"Frames -> Train: {len(train_df)} (efectivos tras resample:
↳ {len(train_ds)}) | Val: {len(val_df)}")
312 print("Parametros activos:\n" + active_params_text(cfg))
313
314 # Pesos por direccion para la L1 ponderada (que BACKWARD sobreviva), SUAVIZADOS
315 # por class_weight_power (1=inverso a freq, 0.5=raiz, 0=uniforme) para no
↳ aplastar STOP.
316 dir_weights = compute_class_weights(train_ds.resampled_frames()) if
↳ cfg.use_class_weights else torch.ones(3)
317 power = getattr(cfg, "class_weight_power", 1.0)
318 if power != 1.0:
319     dir_weights = dir_weights ** power
320 print(f"Pesos L1 por direccion (power={power}) {[DIR_INV[i] for i in range(3)}]:
↳ {dir_weights.tolist()}")
321
322 train_loader = DataLoader(train_ds, batch_size=cfg.batch_size, shuffle=True,
323                             num_workers=cfg.num_workers, pin_memory=True)
324 val_loader = DataLoader(val_ds, batch_size=cfg.batch_size, shuffle=False,
325                             num_workers=cfg.num_workers, pin_memory=True)
326
327 model = SignedControlNet(
328     neck_hidden=tuple(cfg.neck_hidden), dropout=cfg.dropout,
329     ↳ backbone_frozen=cfg.backbone_frozen,
330     lr=cfg.lr, weight_decay=cfg.weight_decay,
331     scheduler_step=cfg.scheduler_step, scheduler_gamma=cfg.scheduler_gamma,
332     dir_weights=dir_weights,
333 )
334
335 # Seleccion del mejor checkpoint segun select_by. make_trainer ya viene con F1
336 ↳ (max);
337 # solo lo cambiamos a MAE si se pide.
338 trainer, csv_logger, checkpoint_callback = make_trainer(cfg, out,
339     ↳ ckpt_prefix="motorsigned")
340 if select_by == "mae":
341     checkpoint_callback.monitor = "val_signed_mae_avg"
342     checkpoint_callback.mode = "min"
343     checkpoint_callback.filename =
344     ↳ "motorsigned-{epoch:02d}-{val_signed_mae_avg:.2f}"
345 else: # "f1" -> usa el default de make_trainer (val_dir_f1_macro, max)
346     checkpoint_callback.filename =
347     ↳ "motorsigned-{epoch:02d}-{val_dir_f1_macro:.3f}"
348 print(f"Seleccion de mejor epoca por: {select_by} "
349     f"({checkpoint_callback.monitor}, mode={checkpoint_callback.mode})")
350
351 print(f"\n[run] ENTRENANDO {tag} (SignedControlNet) [run]")
352 trainer.fit(model, train_loader, val_loader)
353
354 plot_training_progress(os.path.join(csv_logger.log_dir, "metrics.csv"),
355     out_png=os.path.join(out, "training_progress.png"),
356     ↳ cfg=cfg)
357 best = checkpoint_callback.best_model_path
358 if best and os.path.exists(best):
359     print(f"\nEvaluando mejor checkpoint: {best}")
360     model = SignedControlNet.load_from_checkpoint(best, dir_weights=dir_weights)
361 device = "cuda" if torch.cuda.is_available() else "cpu"
362 evaluate_and_plot(model, val_loader, device,
363     out_png=os.path.join(out, "eval_confusion.png"),

```

```

358         report_path=os.path.join(out, "report.txt"), cfg=cfg)
359     print(f"\n[OK] {tag} listo. Resultados en {out}/")

```

Listing 8: nn_signed.py — SignedControlNet, L1 ponderada y metricas

Las piezas propias de este motor

- `signed_to_dir(v, eps)`: decodifica un PWM con signo a {STOP, FWD, BWD} con un umbral ($|v| < \text{eps}$ es STOP). Se usa para poder armar matrices de confusión y un F1 comparable con los demás modelos.
- `SignedDriveDataset`: como el per-frame pero el target es `signed_speed`; el flip intercambia $A \leftrightarrow B$ y **el signo no cambia** (adelante/atrás es a lo largo del eje, el espejo no lo invierte).
- `SignedControlNet`: 2 salidas `tanh` (acotan a $[-1, 1]$, el rango del target normalizado). La pérdida es una **L1 ponderada por dirección**: como ya no hay clases, no hay `class_weights`; para que BACKWARD no se muera se pondera el término L1 de cada motor según la rareza de su dirección real (el equivalente a la regresión de los pesos de clase). `on_validation_epoch_end` loguea el MAE-PWM, el F1 decodificado y el recall de full-stop.
- `run_experiment(cfg, tag, select_by)`: igual que el per-frame, pero permite elegir el criterio del mejor checkpoint: por MAE (la métrica física nativa) o por F1 (comparable con 00-05).

El hallazgo central del enfoque con signo

Funcionó para BACKWARD pero **el STOP colapsó**. Una regresión no tiene incentivo para producir exactamente 0: predecir +8 para un STOP real casi no se castiga en L1, y el umbral después lo lee como FORWARD. El umbral *a posteriori* no lo arregla, porque los stops y los avances lentos se solapan en la salida. **El colapso de STOP es estructural de la regresión**, no un problema de ponderación. Esto motivó el modelo híbrido.

11 nn_hybrid.py — la solución: signed + cabeza “is-stop”

El híbrido combina lo mejor de los dos mundos: **una regresión con signo** para el movimiento (magnitud y sentido) **más una cabeza de clasificación binaria dedicada** a una única pregunta: ¿este motor está parado? La clasificación binaria *sí* tiene el incentivo (su pérdida BCE) que la regresión no tenía.

```

1  """
2  nn_hybrid.py -- MOTOR per-frame HIBRIDO: signed regression + cabeza "is-stop" (no es
    ↪ entriypoint).
3
4  =====
5  Arregla el colapso de STOP del modelo signed (model_06/08): la regresion-a-cero no se
6  detecta por umbral, asi que le agregamos una cabeza de CLASIFICACION binaria explicita
7  "este motor esta parado?" entrenada con BCE -- que SI tiene incentivo para detectar
    ↪ el cero.
8
9  Por motor, 2 salidas:
10  - signed (tanh, PWM con signo)          -> direccion+magnitud CUANDO se mueve
11  - is_stop (logit, binaria)              -> esta parado? (la autoridad sobre STOP)
12
13  DECODE:  STOP si sigmoid(is_stop) > 0.5 ; si no, dir = signo(signed), speed =
    ↪ |signed|.
14
15  Loss = L1_ponderada(signed) + lambda_stop * BCE(is_stop, pos_weight por motor).

```



```

15 La BCE usa pos_weight = #no_stop/#stop por motor (motor B casi no para -> pesa mas).
16
17 Metricas (val): F1 macro de direccion decodificada (comparable con 00-08), MAE-PWM,
   ↪ full-stop recall.
18 Seleccion de mejor epoca por F1 macro. Reusa el Dataset signed de nn_signed.
19 """
20
21 import os
22 import numpy as np
23 import pandas as pd
24 import matplotlib.pyplot as plt
25 import seaborn as sns
26 import torch
27 from torch import nn
28 import torch.nn.functional as F
29 from torch.utils.data import DataLoader
30 from torchvision.models import mobilenet_v3_small, MobileNet_V3_Small_Weights
31 import pytorch_lightning as pl
32 from sklearn.metrics import f1_score, confusion_matrix, classification_report
33
34 from config import active_params_text
35 from nn_perframe import (
36     DIR_MAP, DIR_INV, SPEED_SCALE, compute_class_weights, save_config_snapshot,
37     ↪ make_trainer,
38 )
39
40
41 from nn_signed import SignedDriveDataset, signed_to_dir, DIR_EPS_PWM
42
43
44 def decode_hybrid(signed_norm, stop_prob, eps_norm):
45     """STOP si stop_prob>0.5; si no, direccion = signo del PWM. ->
46     ↪ {0:STOP,1:FWD,2:BWD}."""
47     out = signed_to_dir(signed_norm, eps_norm) # 1/2 segun signo (0 si |v|<eps)
48     out[stop_prob > 0.5] = 0 # la cabeza is-stop manda
49     return out
50
51
52 # =====
53 # MODELO: backbone congelado -> cuello -> 4 cabezas (2 signed + 2 is-stop)
54 # =====
55
56 class HybridControlNet(pl.LightningModule):
57     def __init__(self, neck_hidden=(256,), dropout=0.4, backbone_frozen=True,
58                 lr=1e-3, weight_decay=1e-4, scheduler_step=7, scheduler_gamma=0.1,
59                 lambda_stop=1.0, dir_weights=None, stop_pos_weight=None,
60                 eps_norm=DIR_EPS_PWM / SPEED_SCALE):
61         super().__init__()
62         self.save_hyperparameters(ignore=['dir_weights', 'stop_pos_weight'])
63
64         backbone = mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT)
65         feat_dim = backbone.classifier[0].in_features
66         backbone.classifier = nn.Identity()
67         if backbone_frozen:
68             for p in backbone.features.parameters():
69                 p.requires_grad = False
70         self.backbone = backbone
71
72         layers, d = [], feat_dim
73         for h in neck_hidden:
74             layers += [nn.Linear(d, h), nn.Hardswish(), nn.Dropout(dropout)]
75             d = h

```

```

71     self.neck = nn.Sequential(*layers)
72     self.head_spdA = nn.Linear(d, 1)
73     self.head_spdB = nn.Linear(d, 1)
74     self.head_stopA = nn.Linear(d, 1)
75     self.head_stopB = nn.Linear(d, 1)
76
77     if dir_weights is None:
78         dir_weights = torch.ones(3)
79     if stop_pos_weight is None:
80         stop_pos_weight = torch.ones(2)
81     self.register_buffer("dir_weights", dir_weights)
82     self.register_buffer("stop_pos_weight", stop_pos_weight)
83     self._reset_val_store()
84
85     def _reset_val_store(self):
86         self.val_store = {k: [] for k in ["sA", "sB", "stA", "stB", "dirA_t",
87 ↪ "dirB_t",
88                                     "spdA_t", "spdB_t"]}
89
90     def forward(self, x):
91         z = self.neck(self.backbone(x))
92         return (torch.tanh(self.head_spdA(z)).squeeze(1),
93                 torch.tanh(self.head_spdB(z)).squeeze(1),
94                 self.head_stopA(z).squeeze(1),
95                 self.head_stopB(z).squeeze(1))
96
97     def _loss(self, out, tgt):
98         sA, sB, stA, stB = out
99         is_stopA = (tgt["dirA"] == 0).float()
100         is_stopB = (tgt["dirB"] == 0).float()
101         wA, wB = self.dir_weights[tgt["dirA"]], self.dir_weights[tgt["dirB"]]
102         l1 = (wA * (sA - tgt["spdA"]).abs()).mean() + (wB * (sB -
103 ↪ tgt["spdB"]).abs()).mean()
104         bce = (F.binary_cross_entropy_with_logits(stA, is_stopA,
105 ↪ pos_weight=self.stop_pos_weight[0])
106               + F.binary_cross_entropy_with_logits(stB, is_stopB,
107 ↪ pos_weight=self.stop_pos_weight[1]))
108         return l1 + self.hparams.lambda_stop * bce, l1, bce
109
110     def training_step(self, batch, batch_idx):
111         x, tgt = batch
112         loss, l1, bce = self._loss(self(x), tgt)
113         self.log('train_loss', loss, on_step=False, on_epoch=True, prog_bar=True)
114         self.log('train_l1', l1, on_step=False, on_epoch=True)
115         self.log('train_bce', bce, on_step=False, on_epoch=True)
116         return loss
117
118     def on_validation_epoch_start(self):
119         self._reset_val_store()
120
121     def validation_step(self, batch, batch_idx):
122         x, tgt = batch
123         out = self(x)
124         loss, _, _ = self._loss(out, tgt)
125         self.log('val_loss', loss, on_step=False, on_epoch=True, prog_bar=True)
126         sA, sB, stA, stB = out
127         s = self.val_store
128         s["sA"].append(sA.float().cpu()); s["sB"].append(sB.float().cpu())
129         s["stA"].append(torch.sigmoid(stA).cpu());

```

```

126     ↪ s["stB"].append(torch.sigmoid(stB).cpu())
127     s["dirA_t"].append(tgt["dirA"].cpu()); s["dirB_t"].append(tgt["dirB"].cpu())
128     s["spdA_t"].append(tgt["spdA"].cpu()); s["spdB_t"].append(tgt["spdB"].cpu())
129
130     def on_validation_epoch_end(self):
131         s = {k: torch.cat(v).numpy() for k, v in self.val_store.items()}
132         eps = self.hparams.eps_norm
133         pdA = decode_hybrid(s["sA"], s["stA"], eps)
134         pdB = decode_hybrid(s["sB"], s["stB"], eps)
135         tdA, tdB = s["dirA_t"], s["dirB_t"]
136         f1A = f1_score(tdA, pdA, average='macro', labels=[0, 1, 2], zero_division=0)
137         f1B = f1_score(tdB, pdB, average='macro', labels=[0, 1, 2], zero_division=0)
138         maeA = np.abs(s["sA"] - s["spdA_t"]).mean() * SPEED_SCALE
139         maeB = np.abs(s["sB"] - s["spdB_t"]).mean() * SPEED_SCALE
140         true_stop = (tdA == 0) & (tdB == 0)
141         pred_stop = (pdA == 0) & (pdB == 0)
142         fs = float((pred_stop & true_stop).sum() / true_stop.sum()) if
143         ↪ true_stop.sum() else 0.0
144         self.log('val_signed_mae_A', maeA); self.log('val_signed_mae_B', maeB)
145         self.log('val_signed_mae_avg', (maeA + maeB) / 2, prog_bar=True)
146         self.log('val_f1_A', f1A); self.log('val_f1_B', f1B)
147         self.log('val_dir_f1_macro', (f1A + f1B) / 2, prog_bar=True)
148         self.log('val_fullstop_recall', fs, prog_bar=True)
149
150     def configure_optimizers(self):
151         opt = torch.optim.AdamW(filter(lambda p: p.requires_grad, self.parameters()),
152                                   lr=self.hparams.lr,
153                                   ↪ weight_decay=self.hparams.weight_decay)
154         sched = torch.optim.lr_scheduler.StepLR(opt,
155         ↪ step_size=self.hparams.scheduler_step,
156                                   gamma=self.hparams.scheduler_gamma)
157         return [opt], [sched]
158
159     # =====
160     # GRAFICADO + EVALUACION
161     # =====
162     def plot_training_progress(metrics_csv, out_png, cfg=None):
163         if not os.path.exists(metrics_csv):
164             print(f"[WARN] No se encontro {metrics_csv}."); return
165         m =
166         ↪ pd.read_csv(metrics_csv).groupby("epoch").mean(numeric_only=True).reset_index()
167         panels = [
168             ("Loss total", [("train_loss", "Train"), ("val_loss", "Val")]),
169             ("F1 macro direccion (Val)", [("val_f1_A", "Motor A"), ("val_f1_B", "Motor
170             ↪ B")]),
171             ("MAE PWM con signo (Val)", [("val_signed_mae_A", "Motor A"),
172             ↪ ("val_signed_mae_B", "Motor B")]),
173             ("Recall full-stop (Val)", [("val_fullstop_recall", "full-stop")]),
174         ]
175         fig, axes = plt.subplots(2, 2, figsize=(14, 9))
176         fig.suptitle("Progreso -- HybridControlNet (signed + is-stop)", fontsize=15)
177         for ax, (title, series) in zip(axes.ravel(), panels):
178             plotted = False
179             for col, label in series:
180                 if col in m.columns and m[col].notna().any():
181                     d = m[["epoch", col]].dropna(); ax.plot(d["epoch"], d[col],
182                     ↪ marker="o", label=label); plotted = True
183             ax.set_title(title); ax.set_xlabel("Epoca"); ax.grid(True, alpha=0.3)

```

```

177     ax.legend() if plotted else ax.text(0.5, 0.5, "sin datos", ha="center",
    ↪ va="center", transform=ax.transAxes)
178     fig.text(0.5, 0.005, "Parametros: " + active_params_text(cfg).replace("\n", " |
    ↪ "),
179             ha="center", fontsize=8, family="monospace")
180     plt.tight_layout(rect=[0, 0.03, 1, 1]); plt.savefig(out_png, dpi=150);
    ↪ plt.close(fig)
181     print(f"\n[graf] Curvas guardadas en '{out_png}'")
182
183
184 @torch.no_grad()
185 def evaluate_and_plot(model, loader, device, out_png, report_path=None, cfg=None):
186     model.eval().to(device)
187     eps = model.hparams.eps_norm
188     sA_, sB_, stA_, stB_, tdA_, tdB_, eA, eB = [], [], [], [], [], [], [], []
189     for x, tgt in loader:
190         a, b, sa, sb = model(x.to(device))
191         sA_.append(a.cpu()); sB_.append(b.cpu())
192         stA_.append(torch.sigmoid(sa).cpu()); stB_.append(torch.sigmoid(sb).cpu())
193         tdA_.append(tgt["dirA"]); tdB_.append(tgt["dirB"])
194         eA.append((a.cpu() - tgt["spdA"]).abs()); eB.append((b.cpu() -
    ↪ tgt["spdB"]).abs())
195     sA = torch.cat(sA_.numpy()); sB = torch.cat(sB_.numpy())
196     stA = torch.cat(stA_.numpy()); stB = torch.cat(stB_.numpy())
197     tdA = torch.cat(tdA_.numpy()); tdB = torch.cat(tdB_.numpy())
198     pdA = decode_hybrid(sA, stA, eps); pdB = decode_hybrid(sB, stB, eps)
199     maeA = round(float(torch.cat(eA).mean()) * SPEED_SCALE, 2)
200     maeB = round(float(torch.cat(eB).mean()) * SPEED_SCALE, 2)
201     names = [DIR_INV[i] for i in range(3)]
202     true_stop = (tdA == 0) & (tdB == 0); pred_stop = (pdA == 0) & (pdB == 0)
203     fs = float((pred_stop & true_stop).sum() / true_stop.sum()) if true_stop.sum()
    ↪ else 0.0
204
205     lines = ["==== MOTOR A (IZQ) -- direccion (is-stop + signo) ====="]
206     lines.append(classification_report(tdA, pdA, labels=[0, 1, 2],
    ↪ target_names=names, zero_division=0))
207     lines.append(f"MAE PWM con signo A: {maeA}")
208     lines.append("\n==== MOTOR B (DER) -- direccion (is-stop + signo) =====")
209     lines.append(classification_report(tdB, pdB, labels=[0, 1, 2],
    ↪ target_names=names, zero_division=0))
210     lines.append(f"MAE PWM con signo B: {maeB}")
211     lines.append(f"\nRecall full-stop conjunto (ambos motores STOP): {fs:.3f}")
212     lines.append(f"MAE PWM con signo PROMEDIO: {round((maeA + maeB) / 2, 2)}")
213     report = "\n".join(lines)
214     print("\n" + report)
215     if report_path:
216         open(report_path, "w").write(report + "\n"); print(f"[txt] Reporte guardado
    ↪ en '{report_path}'")
217
218     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
219     for ax, (p, t, title) in zip(axes, [(pdA, tdA, "Motor A (IZQ)", (pdB, tdB,
    ↪ "Motor B (DER)"))]):
220         sns.heatmap(confusion_matrix(t, p, labels=[0, 1, 2]), annot=True, fmt='d',
    ↪ cmap='Blues',
221                     xticklabels=names, yticklabels=names, ax=ax)
222         ax.set_title(f"Confusion -- {title}"); ax.set_xlabel("Predicho");
    ↪ ax.set_ylabel("Real")
223     fig.suptitle("model_09_hybrid -- direccion = is-stop + signo del PWM")
224     plt.tight_layout(rect=[0, 0.04, 1, 1]); plt.savefig(out_png, dpi=150);

```

```

    plt.close(fig)
225     print(f"[graf] Matrices de confusion guardadas en '{out_png}')"
226
227
228     # =====
229     # PIPELINE
230     # =====
231     def run_experiment(cfg, tag, results_dir="results"):
232         torch.set_float32_matmul_precision('medium')
233         out = os.path.join(results_dir, tag)
234         os.makedirs(out, exist_ok=True)
235         save_config_snapshot(cfg, os.path.join(out, "config_used.txt"),
236                             extra=f"# {tag} -- PER-FRAME HIBRIDO signed + is-stop
    ↪ (nn_hybrid.py)")
237
238         print(f"=== {tag} | HIBRIDO === Cargando CSVs...")
239         train_df = pd.read_csv(cfg.train_csv); val_df = pd.read_csv(cfg.val_csv)
240         train_ds = SignedDriveDataset(train_df, is_train=True, cfg=cfg)
241         val_ds = SignedDriveDataset(val_df, is_train=False, cfg=cfg)
242         print(f"Frames -> Train: {len(train_df)} (efectivos: {len(train_ds)}) | Val:
    ↪ {len(val_df)}")
243         print("Parametros activos:\n" + active_params_text(cfg))
244
245         # pesos L1 por direccion (suavizados) + pos_weight de la BCE is-stop por motor.
246         frames = train_ds.resampled_frames()
247         dir_weights = compute_class_weights(frames) if cfg.use_class_weights else
    ↪ torch.ones(3)
248         power = getattr(cfg, "class_weight_power", 1.0)
249         if power != 1.0:
250             dir_weights = dir_weights ** power
251         stop_pw = []
252         for col in ["behaviorA", "behaviorB"]:
253             p_stop = (frames[col] == "STOP").mean()
254             stop_pw.append((1 - p_stop) / p_stop if p_stop > 0 else 1.0)
255         cap = getattr(cfg, "stop_pos_weight_cap", 0.0)
256         if cap and cap > 0:
257             stop_pw = [min(x, cap) for x in stop_pw] # tope para que el motor B no
    ↪ sobre-pare
258         stop_pos_weight = torch.tensor(stop_pw, dtype=torch.float32)
259         print(f"Pesos L1 dir (power={power}): {[round(x,2) for x in
    ↪ dir_weights.tolist()]} | "
260             f"pos_weight is-stop [A,B]: {[round(x,2) for x in stop_pw]}")
261
262         train_loader = DataLoader(train_ds, batch_size=cfg.batch_size, shuffle=True,
263                                   num_workers=cfg.num_workers, pin_memory=True)
264         val_loader = DataLoader(val_ds, batch_size=cfg.batch_size, shuffle=False,
265                                 num_workers=cfg.num_workers, pin_memory=True)
266
267         model = HybridControlNet(
268             neck_hidden=tuple(cfg.neck_hidden), dropout=cfg.dropout,
    ↪ backbone_frozen=cfg.backbone_frozen,
269             lr=cfg.lr, weight_decay=cfg.weight_decay,
270             scheduler_step=cfg.scheduler_step, scheduler_gamma=cfg.scheduler_gamma,
271             lambda_stop=cfg.lambda_stop, dir_weights=dir_weights,
    ↪ stop_pos_weight=stop_pos_weight,
272         )
273
274         # mejor epoca por F1 macro (default de make_trainer).
275         trainer, csv_logger, checkpoint_callback = make_trainer(cfg, out,

```

```

276     ↪ ckpt_prefix="motorhybrid")
checkpoint_callback.filename = "motorhybrid-{epoch:02d}-{val_dir_f1_macro:.3f}"
277     print("Selección de mejor época por: f1 (val_dir_f1_macro, max)")
278
279     print(f"\n[run] ENTRENANDO {tag} (HybridControlNet) [run]")
280     trainer.fit(model, train_loader, val_loader)
281
282     plot_training_progress(os.path.join(csv_logger.log_dir, "metrics.csv"),
283                           out_png=os.path.join(out, "training_progress.png"),
284                           ↪ cfg=cfg)
best = checkpoint_callback.best_model_path
285     if best and os.path.exists(best):
286         print(f"\nEvaluando mejor checkpoint: {best}")
287         model = HybridControlNet.load_from_checkpoint(best, dir_weights=dir_weights,
288                                                       stop_pos_weight=stop_pos_weight)
289     device = "cuda" if torch.cuda.is_available() else "cpu"
290     evaluate_and_plot(model, val_loader, device,
291                      out_png=os.path.join(out, "eval_confusion.png"),
292                      report_path=os.path.join(out, "report.txt"), cfg=cfg)
293     print(f"\n[OK] {tag} listo. Resultados en {out}/")

```

Listing 9: nn_hybrid.py — HybridControlNet, decode con compuerta y pos_weight por motor

Cómo funciona

- `decode_hybrid(signed, stop_prob, eps)`: la **compuerta**. Si la cabeza is-stop supera 0.5, el motor está STOP; si no, la dirección la da el signo del PWM. La cabeza de stop *manda* sobre el signo.
- `HybridControlNet`: 4 cabezas por imagen — 2 de regresión `tanh` (signed A y B) y 2 de clasificación binaria (is-stop A y B). La pérdida es L1 ponderada(signed) + $\lambda_{\text{stop}} \cdot \text{BCE}(\text{is-stop})$. La BCE usa un **pos_weight por motor** (la fracción no-stop / stop): como el motor B casi nunca para, su peso positivo es alto ($\sim 6.5\times$) para que la cabeza “vea” los pocos stops.
- `run_experiment`: calcula los pesos L1 (suavizados por `class_weight_power`) y el `pos_weight` por motor (con un tope opcional `stop_pos_weight_cap`, que resultó ser el knob clave para que el motor B no *sobre-pare*). Selecciona el mejor checkpoint por F1.

Qué demostró el híbrido

La cabeza is-stop explícita **recupera el STOP**: el recall de full-stop pasó de ~ 0.02 (signed) a $\sim 0.21\text{--}0.30$ (híbrido). El problema dejó de ser estructural y pasó a ser de *calibración* (cuánto pesar esa cabeza), que es exactamente lo que después optimizó el sweep.

Part IV

Entrypoints y experimentación

12 La convención `model_NN_*`: un archivo por modelo

Cada experimento concreto vive en su propio archivo `model_NN_<nombre>.py` con sus hiperparámetros **congelados** adentro (visibles al abrirlo) y llama a `run_experiment(CFG, "model_NN_...")` del motor correspondiente. Así “se ve el modelo en el código” y los resultados caen en `results/model_NN/`. Los archivos son cortos porque toda la maquinaria está en los motores. Veamos tres representativos.

12.1 El baseline no-deep: model_00_baseline.py

Antes de cualquier red, conviene un **piso** tonto contra el cual comparar. Si la red sofisticada no le gana a esto, algo anda mal.

```

1  """
2  model_00_baseline.py -- Modelo 0: BASELINE no-deep (Random Forest).
3  =====
4  El PISO contra el que se contrastan los modelos deep. Misma tarea:
5  IMAGEN -> direccion {STOP,FORWARD,BACKWARD} + velocidad, por motor (A y B).
6
7  Random Forest sobre imagenes en grises 32x32 aplanadas:
8      - 2 clasificadores de direccion (motor A, motor B), class_weight='balanced'.
9      - 2 regresores de velocidad (motor A, motor B), MAE como metrica.
10 Split por sesion ya materializado: dataset_train.csv / dataset_val.csv.
11
12 Hiperparametros CONGELADOS abajo (esto ES el modelo). NO usa Lightning ni GPU.
13 Correr: .venv/bin/python model_00_baseline.py -> resultados en
14     ↪ results/model_00_baseline/
15 """
16
17 import os
18 import time
19 import numpy as np
20 import pandas as pd
21 import cv2
22 import matplotlib.pyplot as plt
23 import seaborn as sns
24
25 from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
26 from sklearn.metrics import classification_report, confusion_matrix,
27     ↪ mean_absolute_error
28
29 # ===== hiperparametros CONGELADOS de este modelo =====
30 TRAIN_CSV = "dataset_train.csv"
31 VAL_CSV = "dataset_val.csv"
32 IMG_SIZE = (32, 32) # imagenes en grises, aplanadas a 1024 features
33 N_ESTIMATORS = 100
34 MAX_DEPTH = 15
35 RANDOM_STATE = 42
36 TAG = "model_00_baseline"
37 LABELS = ["STOP", "FORWARD", "BACKWARD"]
38
39
40 def load_dataset(csv_path, img_size=IMG_SIZE):
41     """Carga imagenes (grises, aplanadas) y los targets por motor."""
42     df = pd.read_csv(csv_path)
43     X, dirA, dirB, spdA, spdB = [], [], [], [], []
44     print(f"Cargando {len(df)} imagenes de {csv_path} a {img_size}...")
45     start = time.time()
46     for _, row in df.iterrows():
47         if not os.path.exists(row['image_path']):
48             continue
49         img = cv2.imread(row['image_path'], cv2.IMREAD_GRAYSCALE)
50         X.append(cv2.resize(img, img_size).flatten())
51         dirA.append(row['behaviorA']); dirB.append(row['behaviorB'])
52         spdA.append(float(row['speedA'])); spdB.append(float(row['speedB']))
53     print(f" -> {len(X)} cargadas en {time.time() - start:.1f}s")
54     return (np.array(X), np.array(dirA), np.array(dirB),
55           np.array(spdA), np.array(spdB))

```



```

54 def run_experiment(results_dir="results"):
55     out = os.path.join(results_dir, TAG)
56     os.makedirs(out, exist_ok=True)
57     with open(os.path.join(out, "config_used.txt"), "w") as f:
58         f.write(f"# {TAG} -- BASELINE no-deep (model_00_baseline.py)\n\n")
59         f.write(f"modelo = RandomForest (clasif. direccion + regresion velocidad, por
↳ motor)\n")
60         f.write(f"img_size = {IMG_SIZE} (grises, aplanadas)\n")
61         f.write(f"n_estimators = {N_ESTIMATORS}\nmax_depth = {MAX_DEPTH}\n")
62         f.write(f"class_weight = balanced (direccion)\nrandom_state =
↳ {RANDOM_STATE}\n")
63
64     X_tr, dirA_tr, dirB_tr, spdA_tr, spdB_tr = load_dataset(TRAIN_CSV)
65     X_va, dirA_va, dirB_va, spdA_va, spdB_va = load_dataset(VAL_CSV)
66     print(f"\nSplit por sesion -> Train: {len(X_tr)} | Val: {len(X_va)}")
67
68     report_lines = []
69     cms = {}
70     # --- DIRECCION: un Random Forest por motor ---
71     for name, y_tr, y_va in [("A (IZQ)", dirA_tr, dirA_va), ("B (DER)", dirB_tr,
↳ dirB_va)]:
72         print(f"\nEntrenando RF direccion Motor {name}...")
73         clf = RandomForestClassifier(n_estimators=N_ESTIMATORS, max_depth=MAX_DEPTH,
74                                     random_state=RANDOM_STATE, n_jobs=-1,
75                                     class_weight='balanced')
76         clf.fit(X_tr, y_tr)
77         y_pred = clf.predict(X_va)
78         rep = classification_report(y_va, y_pred, labels=LABELS, zero_division=0)
79         report_lines.append(f"===== DIRECCION MOTOR {name} =====\n{rep}")
80         cms[name] = confusion_matrix(y_va, y_pred, labels=LABELS)
81         print(rep)
82
83     # --- VELOCIDAD: un regresor por motor ---
84     for name, y_tr, y_va in [("A (IZQ)", spdA_tr, spdA_va), ("B (DER)", spdB_tr,
↳ spdB_va)]:
85         print(f"\nEntrenando RF velocidad Motor {name}...")
86         reg = RandomForestRegressor(n_estimators=N_ESTIMATORS, max_depth=MAX_DEPTH,
87                                     random_state=RANDOM_STATE, n_jobs=-1)
88         reg.fit(X_tr, y_tr)
89         mae = mean_absolute_error(y_va, reg.predict(X_va))
90         report_lines.append(f"MAE velocidad Motor {name}: {mae:.2f} (escala 0-100)")
91         print(f"MAE velocidad Motor {name}: {mae:.2f}")
92
93     # guardar report.txt
94     report = "\n".join(report_lines)
95     with open(os.path.join(out, "report.txt"), "w") as f:
96         f.write(report + "\n")
97
98     # guardar matrices de confusion por motor (mismo formato que los modelos deep)
99     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
100    for ax, name in zip(axes, ["A (IZQ)", "B (DER)"]):
101        sns.heatmap(cms[name], annot=True, fmt='d', cmap='Blues',
102                    xticklabels=LABELS, yticklabels=LABELS, ax=ax)
103        ax.set_title(f"Confusion -- Motor {name}"); ax.set_xlabel("Predicho");
↳ ax.set_ylabel("Real")
104    fig.suptitle("model_00_baseline -- Random Forest (no-deep)")
105    plt.tight_layout(); plt.savefig(os.path.join(out, "eval_confusion.png"),
↳ dpi=150); plt.close(fig)
106    print(f"\n[OK] {TAG} listo. Resultados en {out}/")

```

```

107
108
109 if __name__ == "__main__":
110     run_experiment()

```

Listing 10: model_00_baseline.py — Random Forest sobre pixeles crudos

Es un **Random Forest** (sklearn, sin deep learning): toma cada imagen en grises 32×32 aplanada (1024 números) y entrena 2 clasificadores de dirección + 2 regresores de velocidad. No entiende de imágenes; solo ve píxeles crudos. Su rol es la fila “0” del scoreboard: deja decir “la CNN da 0.62, el RF da 0.40 → la red aporta valor real”.

12.2 El campeón: model_01_perframe.py

```

1  """
2  model_01_perframe.py -- Modelo 1: PER-FRAME (MobileNetV3-small congelado).
3  =====
4  El MEJOR modelo hasta ahora. F1 macro direccion (val por sesion) = 0.629.
5  Una sola imagen -> 4 cabezas (dirA, dirB, spdA, spdB). Baseline a superar.
6
7  Hiperparametros CONGELADOS abajo (esto ES el modelo). Motor en nn_perframe.py.
8  Correr: .venv/bin/python model_01_perframe.py -> resultados en
9         ↪ results/model_01_perframe/
10 """
11 from config import Config
12 from nn_perframe import run_experiment
13
14 CFG = Config(
15     # arquitectura
16     neck_hidden=[256], dropout=0.4, backbone_frozen=True,
17     # optimizacion
18     batch_size=64, lr=1e-3, weight_decay=1e-4, max_epochs=15,
19     scheduler_step=7, scheduler_gamma=0.1,
20     # balanceo / loss
21     use_class_weights=True, lambda_speed=1.0,
22 )
23
24 if __name__ == "__main__":
25     run_experiment(CFG, "model_01_perframe")

```

Listing 11: model_01_perframe.py — el mejor modelo

Apenas arma una Config congelada (MobileNet congelado, cuello [256], dropout 0.4, 15 épocas) y llama a run_experiment del motor per-frame. Es el mejor de todos (F1 0.62/0.63).

12.3 El híbrido: model_09_hybrid.py

```

1  """
2  model_09_hybrid.py -- Modelo 9: PER-FRAME HIBRIDO (signed + cabeza "is-stop").
3  =====
4  Intenta rescatar el STOP que el enfoque signed (model_06/08) no podia detectar.
5  Por motor: salida signed (tanh, PWM con signo) PARA cuando se mueve + cabeza binaria
6  "is-stop" (BCE) que decide explicitamente si esta parado.
7  DECODE: STOP si is-stop>0.5 ; si no, direccion = signo(signed), speed = |signed|.
8
9  Igual que model_08 (bases signed, arquitectura per-frame, pesos suavizados) + la
10     ↪ cabeza
11 is-stop con su BCE (lambda_stop). Seleccion por F1 macro. Motor en nn_hybrid.py.

```

```

11 Correr: .venv/bin/python model_09_hybrid.py -> results/model_09_hybrid/
12 """
13 from config import Config
14 from nn_hybrid import run_experiment
15
16 CFG = Config(
17     # bases SIGNED (mismas que model_06/08)
18     train_csv="dataset_train_signed.csv",
19     val_csv="dataset_val_signed.csv",
20     # arquitectura/optimizacion (= model_08)
21     neck_hidden=[256], dropout=0.4, backbone_frozen=True,
22     batch_size=64, lr=1e-3, weight_decay=1e-4, max_epochs=15,
23     scheduler_step=7, scheduler_gamma=0.1,
24     # balanceo
25     use_class_weights=True,
26     class_weight_power=0.5, # pesos L1 por direccion suavizados (= model_08)
27     lambda_stop=1.0, # peso de la BCE de la cabeza is-stop
28 )
29
30 if __name__ == "__main__":
31     run_experiment(CFG, "model_09_hybrid")

```

Listing 12: model_09_hybrid.py — signed + is-stop

Misma idea: congela la config (bases con signo, pesos suavizados, `lambda_stop=1`) y delega en el motor híbrido.

12.4 Los diez modelos de un vistazo

El resto de los entrypoints siguen el mismo patrón; cambian solo la `Config` y el motor. La tabla resume el árbol completo de experimentos:

Modelo	Motor	Qué prueba
00 baseline	sklearn	Random Forest sobre píxeles (piso de comparación)
01 per-frame	nn_perframe	MobileNet congelado, 1 imagen → acción. El mejor
02 gru_v1	nn_sequence	Primer temporal (GRU sin regularizar)
03 gru_v2	nn_sequence	GRU regularizado (hidden chico, dropout y wd altos)
04 lstm	nn_sequence	Misma config que 03 con celda LSTM
05 gru_v3_antiredun	nn_sequence	GRU v2 + anti-redundancia de ventanas
06 signed	nn_signed	Target = PWM con signo (selección por MAE)
07 lstm_antiredun	nn_sequence	LSTM + anti-redundancia (completa el grid)
08 signed_f1	nn_signed	Signed con selección por F1 y pesos suavizados
09 hybrid	nn_hybrid	Signed + cabeza is-stop (recupera el STOP)

13 sweep.py — exploración de hiperparámetros

Una vez identificado un modelo prometedor, queremos **optimizar** sus hiperparámetros de forma sistemática. `sweep.py` es un explorador **genérico** (sirve para cualquier motor) que barre una grilla de configuraciones, corre cada una en un **subproceso aislado** (para que no se acumule memoria de GPU entre corridas), y resume todo con una **frontera de Pareto** y un mecanismo **anti-ruido**.

```

1  """
2  sweep.py -- Exploracion de hiperparametros GENERICA (frontera de Pareto + anti-ruido).
3  =====
4  Barre una grilla de hiperparametros sobre CUALQUIER modelo (hybrid / perframe /
5      ↪ signed /
6  sequence), reusando su 'run_experiment'. No pisa los resultados viejos: todo va a
7  'results/sweeps/<nombre>/'. Cada config corre en un SUBPROCESO aislado (evita que la
8  memoria CUDA se acumule entre corridas en la RAM unificada de la Jetson).
9
10 QUE PRODUCE (en results/sweeps/<nombre>):
11 - <NN>_<tag>/          : carpeta de resultados de cada config (eval, report,
12     ↪ metrics, ckpt)
13 - sweep_summary.csv    : una fila por config con sus hiperparametros + metricas
14     ↪ del MEJOR
15                           checkpoint (el que elige el modelo: max
16     ↪ val_dir_f1_macro).
17 - pareto.png           : scatter F1-macro vs full-stop-recall, con la FRONTERA DE
18     ↪ PARETO
19                           marcada (las configs no dominadas: nadie les gana en
20     ↪ ambos ejes).
21 - param_effects.png    : efecto marginal de cada parametro sobre F1 y full-stop.
22 - reseed_summary.csv   : ANTI-RUIDO. Re-corre los finalistas (los de Pareto) con
23     ↪ varias
24                           semillas y reporta media +/- desvio -> la ventaja
25     ↪ sobrevive al
26                           ruido del val (+/-0.05) o era suerte?
27
28 USO:
29 python3 sweep.py          # corre el sweep definido en SWEEP (abajo)
30 python3 sweep.py --run-one ... # (interno) corre UNA config en subprocesso
31
32 La grilla y el modelo objetivo se editan en el dict SWEEP, al final del archivo.
33 """
34
35 import argparse
36 import itertools
37 import json
38 import os
39 import subprocess
40 import sys
41 import time
42 from dataclasses import asdict, replace
43
44 import numpy as np
45 import pandas as pd
46 import matplotlib
47 matplotlib.use("Agg")
48 import matplotlib.pyplot as plt
49
50 from config import Config
51
52 # nombre de motor -> (modulo, funcion run_experiment)
53 ENGINES = {
54     "hybrid": ("nn_hybrid", "run_experiment"),
55     "perframe": ("nn_perframe", "run_experiment"),
56     "signed": ("nn_signed", "run_experiment"),
57     "sequence": ("nn_sequence", "run_experiment"),
58 }

```

```

51
52
53 # =====
54 # WORKER: corre UNA config (en subprocesso aislado)
55 # =====
56 def run_one(engine, tag, results_dir, seed, cfg_json):
57     import importlib
58     import torch_lightning as pl
59     pl.seed_everything(int(seed), workers=True) # semilla global: init, shuffle,
        ↪ augment
60     mod, fn = ENGINES[engine]
61     run = getattr(importlib.import_module(mod), fn)
62     cfg = replace(Config(**json.loads(cfg_json)), seed=int(seed))
63     run(cfg, tag, results_dir=results_dir)
64
65
66 # =====
67 # COLECCION de metricas
68 # =====
69 def find_metrics_csv(run_dir):
70     logs = os.path.join(run_dir, "logs")
71     if not os.path.isdir(logs):
72         return None
73     for v in sorted(os.listdir(logs)):
74         p = os.path.join(logs, v, "metrics.csv")
75         if os.path.exists(p):
76             return p
77     return None
78
79
80 def best_operating_point(metrics_csv, select="val_dir_f1_macro", mode="max"):
81     """Metricas en la epoca del MEJOR checkpoint (el criterio con que el modelo
        ↪ elige)."""
82     if not metrics_csv or not os.path.exists(metrics_csv):
83         return None
84     m = pd.read_csv(metrics_csv)
85     g = m.groupby("epoch").max(numeric_only=True) # junta filas train/val de cada
        ↪ epoca
86     if select not in g or g[select].dropna().empty:
87         return None
88     idx = g[select].idxmax() if mode == "max" else g[select].idxmin()
89     return g.loc[idx].to_dict()
90
91
92 def pareto_mask(xs, ys):
93     """True si el punto NO esta dominado (ambos ejes a MAXIMIZAR)."""
94     xs, ys = np.asarray(xs, float), np.asarray(ys, float)
95     n = len(xs); keep = np.ones(n, bool)
96     for i in range(n):
97         for j in range(n):
98             if i == j:
99                 continue
100             if xs[j] >= xs[i] and ys[j] >= ys[i] and (xs[j] > xs[i] or ys[j] > ys[i]):
101                 keep[i] = False
102             break
103     return keep
104
105
106 # =====

```

```

107 # ORQUESTADOR
108 # =====
109 def _short(k, v):
110     key = "".join(w[0] for w in k.split("_")) # lambda_stop -> ls
111     sv = str(v).replace(" ", "").replace("[", "").replace("]", "").replace(",", "-")
112     return f"{key}{sv}"
113
114
115 def launch(engine, tag, results_dir, seed, cfg):
116     """Corre una config en subprocesso; devuelve (returncode,
117     ↪ metricas-del-mejor-ckpt)."""
118     cfg_json = json.dumps(asdict(cfg))
119     log = os.path.join(results_dir, tag + ".log")
120     os.makedirs(os.path.dirname(log) or ".", exist_ok=True)
121     cmd = [sys.executable, os.path.abspath(__file__), "--run-one",
122           "--engine", engine, "--tag", tag, "--results-dir", results_dir,
123           "--seed", str(seed), "--cfg", cfg_json]
124     with open(log, "w") as f:
125         rc = subprocess.run(cmd, stdout=f, stderr=subprocess.STDOUT).returncode
126         op = best_operating_point(find_metrics_csv(os.path.join(results_dir, tag)))
127     return rc, (op or {})
128
129 METRIC_COLS = ["val_dir_f1_macro", "val_fullstop_recall",
130               "val_signed_mae_avg", "val_signed_mae_A", "val_signed_mae_B",
131               "val_mae_A", "val_mae_B"]
132
133
134 def run_sweep(name, engine, base, grid, axis_x="val_dir_f1_macro",
135              axis_y="val_fullstop_recall", reseed_seeds=(101, 202)):
136     results_dir = os.path.join("results", "sweeps", name)
137     os.makedirs(results_dir, exist_ok=True)
138     keys = list(grid)
139     combos = [dict(zip(keys, vals)) for vals in itertools.product(*[grid[k] for k in
140     ↪ keys])]
141     print(f"[sweep {name}] engine={engine} | {len(combos)} configs | ejes Pareto:
142     ↪ {axis_x} vs {axis_y}",
143           flush=True)
144
145     records = []
146     for i, ov in enumerate(combos):
147         tag = f"{i:02d}_" + "_".join(_short(k, ov[k]) for k in keys)
148         cfg = replace(base, **ov)
149         t0 = time.time()
150         print(f"[sweep] ({i+1}/{len(combos)}) {tag} ...", flush=True)
151         rc, op = launch(engine, tag, results_dir, base.seed, cfg)
152         rec = {"tag": tag, "seed": base.seed, "rc": rc, "secs": round(time.time() -
153         ↪ t0)}
154         rec.update({k: ov[k] for k in keys})
155         rec.update({c: op.get(c) for c in METRIC_COLS if c in op})
156         rec["_override"] = ov
157         records.append(rec)
158         _dump_summary(records, keys, os.path.join(results_dir, "sweep_summary.csv"))
159         print(f"    -> rc={rc} {axis_x}={op.get(axis_x)} {axis_y}={op.get(axis_y)}
160         ↪ ({rec['secs']}s)",
161               flush=True)
162
163     df = _dump_summary(records, keys, os.path.join(results_dir, "sweep_summary.csv"))
164     ok = df.dropna(subset=[axis_x, axis_y]).reset_index(drop=True)

```

```

161     if ok.empty:
162         print("[sweep] ninguna config produjo metricas; abortando plots."); return
163     ok = ok.copy()
164     ok["pareto"] = pareto_mask(ok[axis_x], ok[axis_y])
165     _dump_summary(records, keys, os.path.join(results_dir, "sweep_summary.csv"),
166     ↪ pareto_df=ok)
167     plot_pareto(ok, axis_x, axis_y, os.path.join(results_dir, "pareto.png"), name)
168     plot_param_effects(ok, keys, axis_x, axis_y, os.path.join(results_dir,
169     ↪ "param_effects.png"))
170
171     # ----- ANTI-RUIDO: reseed de los finalistas (frontera de Pareto) -----
172     finalists = [records[i] for i in ok.index[ok["pareto"]].tolist()]
173     print(f"[sweep] reseed anti-ruido de {len(finalists)} finalistas con seeds
174     ↪ {reseed_seeds}", flush=True)
175     rs = []
176     for rec in finalists:
177         ov = rec["_override"]
178         vals = {axis_x: [rec.get(axis_x)], axis_y: [rec.get(axis_y)]}
179         for s in reseed_seeds:
180             tag = rec["tag"] + f"_seed{s}"
181             _, op = launch(engine, tag, results_dir, s, replace(base, **ov, seed=s))
182             vals[axis_x].append(op.get(axis_x)); vals[axis_y].append(op.get(axis_y))
183         row = {"tag": rec["tag"], "n_seeds": 1 + len(reseed_seeds)}
184         for ax in (axis_x, axis_y):
185             v = np.array([x for x in vals[ax] if x is not None], float)
186             row[f"{ax}_mean"] = round(float(v.mean()), 4) if len(v) else None
187             row[f"{ax}_std"] = round(float(v.std()), 4) if len(v) else None
188         rs.append(row)
189         pd.DataFrame(rs).to_csv(os.path.join(results_dir, "reseed_summary.csv"),
190         ↪ index=False)
191         print(f"      {row['tag']}:
192         ↪ {axis_x}={row.get(axis_x+'_mean')}+/-{row.get(axis_x+'_std')} "
193         ↪ f"{axis_y}={row.get(axis_y+'_mean')}+/-{row.get(axis_y+'_std')}",
194         ↪ flush=True)
195     print(f"\n[sweep {name}] LISTO. Resumen en {results_dir}/", flush=True)
196
197 def _dump_summary(records, keys, path, pareto_df=None):
198     cols = ["tag", "seed", "rc", "secs"] + keys + [c for c in METRIC_COLS
199     ↪ if any(c in r for r in records)]
200     df = pd.DataFrame([k: r.get(k) for k in cols for r in records])
201     if pareto_df is not None:
202         df = df.merge(pareto_df[["tag", "pareto"]], on="tag", how="left")
203     df.to_csv(path, index=False)
204     return df
205
206 # =====
207 # GRAFICOS
208 # =====
209 def plot_pareto(df, ax_x, ax_y, out_png, name):
210     fig, ax = plt.subplots(figsize=(9, 7))
211     dom = df[~df["pareto"]]; par = df[df["pareto"]].sort_values(ax_x)
212     ax.scatter(dom[ax_x], dom[ax_y], c="#9aa0a6", s=45, label="dominadas", zorder=2)
213     ax.scatter(par[ax_x], par[ax_y], c="#d62728", s=90, edgecolor="k",
214     ↪ label="frontera de Pareto", zorder=3)
215     ax.plot(par[ax_x], par[ax_y], "--", c="#d62728", alpha=0.6, zorder=1)
216     for _, r in par.iterrows():
217         ax.annotate(r["tag"], (r[ax_x], r[ax_y]), fontsize=7, xytext=(4, 4),

```



```

214         textcoords="offset points")
215     ax.set_xlabel(ax_x + " (-> mejor)"); ax.set_ylabel(ax_y + " (~ mejor)")
216     ax.set_title(f"Frontera de Pareto -- sweep '{name}'\n(cada punto = una config, en
    ↪ su mejor checkpoint)")
217     ax.grid(True, alpha=0.3); ax.legend()
218     plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.close(fig)
219     print(f"[graf] Pareto guardado en '{out_png}'")
220
221
222 def plot_param_effects(df, keys, ax_x, ax_y, out_png):
223     n = len(keys)
224     fig, axes = plt.subplots(1, n, figsize=(5 * n, 4.5), squeeze=False)
225     for ax, k in zip(axes[0], keys):
226         d = df.copy(); d[k] = d[k].astype(str)
227         g = d.groupby(k)[[ax_x, ax_y]].mean().reset_index()
228         x = np.arange(len(g))
229         ax.plot(x, g[ax_x], "o-", color="#1f77b4", label=ax_x)
230         ax2 = ax.twinx()
231         ax2.plot(x, g[ax_y], "s--", color="#d62728", label=ax_y)
232         ax.set_xticks(x); ax.set_xticklabels(g[k], rotation=20)
233         ax.set_xlabel(k); ax.set_ylabel(ax_x, color="#1f77b4"); ax2.set_ylabel(ax_y,
    ↪ color="#d62728")
234         ax.set_title(f"efecto de {k}"); ax.grid(True, alpha=0.3)
235     fig.suptitle("Efecto marginal de cada parametro (promediando el resto)",
    ↪ fontsize=13)
236     plt.tight_layout(rect=[0, 0, 1, 0.96]); plt.savefig(out_png, dpi=150);
    ↪ plt.close(fig)
237     print(f"[graf] Efectos por parametro en '{out_png}'")
238
239
240 # =====
241 # DEFINICION DEL SWEEP (edita aca)
242 # =====
243 def model09_sweep():
244     base = Config(
245         train_csv="dataset_train_signed.csv", val_csv="dataset_val_signed.csv",
246         neck_hidden=[256], dropout=0.4, backbone_frozen=True,
247         batch_size=64, lr=1e-3, weight_decay=1e-4, max_epochs=15,
248         scheduler_step=7, scheduler_gamma=0.1,
249         use_class_weights=True, class_weight_power=0.5, lambda_stop=1.0,
250     )
251     grid = {
252         # Tier 1 (knobs de la falla) + Tier 2
253         ↪ (capacidad)
254         "lambda_stop": [0.3, 0.5, 1.0], # peso de la BCE is-stop (barre el
255         ↪ Pareto)
256         "stop_pos_weight_cap": [2.0, 3.5, 6.5], # tope al pos_weight del motor B
257         ↪ (sobre-stop)
258         "neck_hidden": [[256], [512, 256]], # capacidad del cuello
259         "dropout": [0.4, 0.5], # regularizacion
260         # 3*3*2*2 = 36 configs
261     }
262     return dict(name="m09_tier2", engine="hybrid", base=base, grid=grid,
263         axis_x="val_dir_f1_macro", axis_y="val_fullstop_recall",
264         reseed_seeds=(101, 202))
265
266
267 def model01_sweep():
268     """Sweep del per-frame (model_01, el mejor): arquitectura + regularizacion +
269     ↪ optimizacion."""
270     base = Config(
271         # = model_01_perframe (bases

```

```

↪ dir+speed por defecto)
265     neck_hidden=[256], dropout=0.4, backbone_frozen=True,
266     batch_size=64, lr=1e-3, weight_decay=1e-4, max_epochs=15,
267     scheduler_step=7, scheduler_gamma=0.1,
268     use_class_weights=True, lambda_speed=1.0,
269 )
270 grid = {
271     "neck_hidden": [[256], [512, 256]],           # capacidad del cuello
272     "dropout": [0.3, 0.4, 0.5],                  # regularizacion
273     "lr": [5e-4, 1e-3],                          # optimizacion
274     "weight_decay": [1e-4, 1e-3],                 # regularizacion L2
275 }                                                  # 2*3*2*2 = 24 configs
276 return dict(name="m01_perframe", engine="perframe", base=base, grid=grid,
277             axis_x="val_dir_f1_macro", axis_y="val_fullstop_recall",
278             reseed_seeds=(101, 202))
279
280
281 SWEEPS = {"m09": model09_sweep, "m01": model01_sweep}
282
283
284 def smoke_sweep():
285     """Mini-sweep para validar el pipeline end-to-end (2 configs, 2 epocas, 1
286     ↪ reseed)."""
287     spec = model09_sweep()
288     spec["base"] = replace(spec["base"], max_epochs=2)
289     spec["grid"] = {"lambda_stop": [0.5, 1.0]}
290     spec["name"] = "smoke"
291     spec["reseed_seeds"] = (101,)
292     return spec
293
294 def main():
295     ap = argparse.ArgumentParser()
296     ap.add_argument("--run-one", action="store_true")
297     ap.add_argument("--smoke", action="store_true")
298     ap.add_argument("--which", default="m09", choices=list(SWEEPS)) # que sweep
299     ↪ correr
300     ap.add_argument("--engine"); ap.add_argument("--tag");
301     ↪ ap.add_argument("--results-dir")
302     ap.add_argument("--seed"); ap.add_argument("--cfg")
303     args = ap.parse_args()
304     if args.run_one:
305         run_one(args.engine, args.tag, args.results_dir, args.seed, args.cfg)
306     elif args.smoke:
307         run_sweep(**smoke_sweep())
308     else:
309         run_sweep(**SWEEPS[args.which]())
310
311 if __name__ == "__main__":
312     main()

```

Listing 13: sweep.py — grilla, subprocesos aislados, Pareto y reseed anti-ruido

Las ideas centrales

- **Subproceso por config:** launch arma un comando `python sweep.py --run-one ...` con la config serializada en JSON; `run_one` la reconstruye, fija la semilla y llama al `run_experiment`

del motor. Aislar cada corrida evita que un crash o una fuga de memoria contaminen al resto.

- **Operating point honesto:** `best_operating_point` lee el `metrics.csv` de cada corrida y toma las métricas en la época del *mejor checkpoint* (el que el modelo realmente desplegaría), no el pico de cualquier métrica suelta.
- **Frontera de Pareto:** `pareto_mask` marca las configs *no dominadas* (aquellas a las que nadie les gana en ambos ejes a la vez). Acá los ejes son F1 macro vs recall de full-stop, porque — y esto es un punto fino — **el F1 solo no protege el frenado**: de hecho rankearía más alto a un modelo que no frena. La frontera muestra el *trade-off* en vez de esconderlo en un número.
- **Anti-ruido (reseed):** el val es chico y la métrica tiembla (± 0.05). Tras la grilla, se re-corren los finalistas de Pareto con **varias semillas** y se reporta media $\pm$ desvío. Así se distingue una mejora real de una casualidad afortunada. (En la práctica, esto evitó que creyéramos picos optimistas de una sola semilla.)
- `plot_pareto` y `plot_param_effects` generan las figuras: la frontera y el efecto marginal de cada parámetro. `model09_sweep` y `model01_sweep` definen las grillas concretas; `smoke_sweep` es una versión mínima (2 configs, 2 épocas) para validar el pipeline rápido antes de gastar horas.

Part V

Resultados y conclusión

14 El scoreboard

Métrica principal: **F1 macro de dirección por motor** (val por sesión). Velocidad: MAE.

#	Modelo	F1 macro (A / B)	full-stop
00	baseline RF	0.40 / 0.23	—
01	per-frame	0.62 / 0.63	decente
02	GRU v1	0.45 / 0.57	—
03	GRU v2	0.50 / 0.62	—
04	LSTM	0.51 / 0.58	—
05	GRU v3 anti-redun	0.50 / 0.60	—
06	signed (sel. MAE)	0.43 / 0.54	0.025
07	LSTM anti-redun	0.52 / 0.56	—
08	signed (sel. F1)	0.42 / 0.50	0.025
09	híbrido	0.45 / 0.33	0.21
<i>Sweeps (mejor config, reseed con 3 semillas):</i>			
	per-frame afinado	0.62 ± 0.01	0.42 ± 0.13
	híbrido afinado	0.53 ± 0.02	0.30 ± 0.07

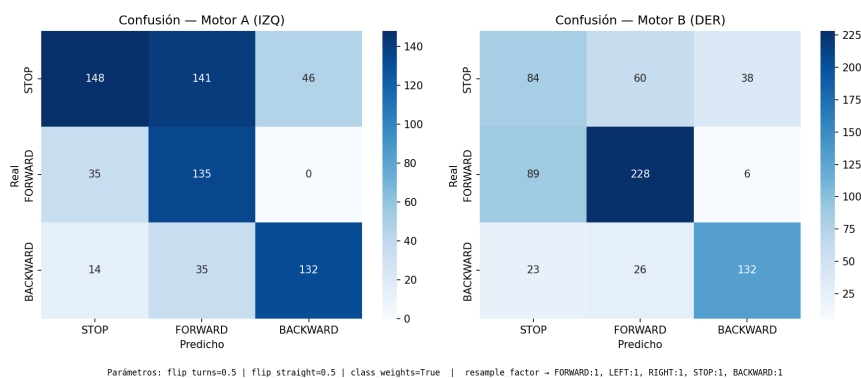


Figure 1: Matrices de confusión por motor del mejor modelo (model_01, per-frame). La diagonal fuerte muestra que clasifica bien las tres direcciones; el error dominante es STOP↔FORWARD, el borde físicamente ambiguo desde un solo frame.

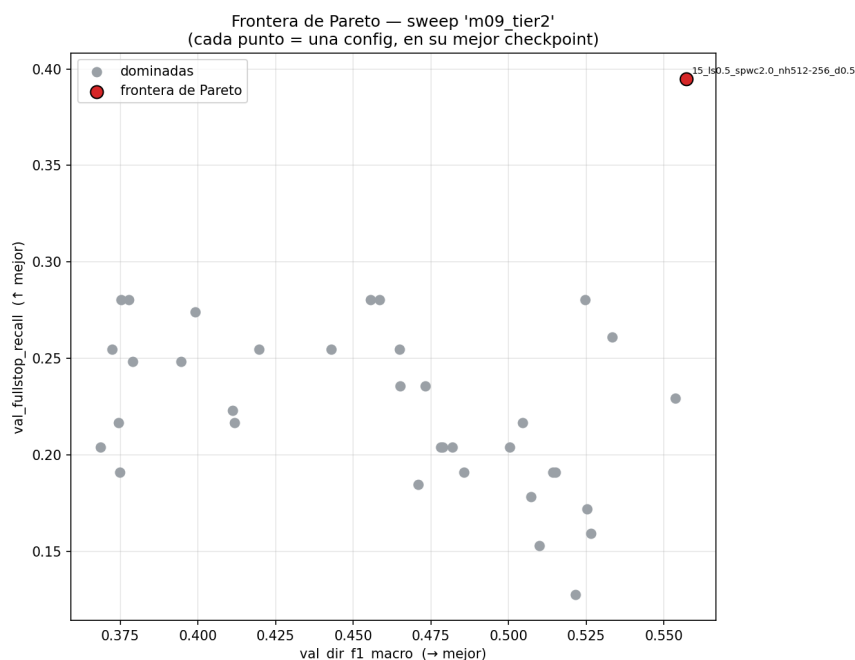


Figure 2: Frontera de Pareto del sweep del híbrido (F1 macro vs recall de full-stop). Cada punto es una config en su mejor checkpoint; en rojo, las no dominadas. La config ganadora mejora al híbrido original en ambos ejes.

15 Conclusión: el arco completo

El proyecto recorrió, de lo simple a lo complejo: un **baseline** tonto (RF) → el **per-frame** (mejor) → modelos **temporales** (GRU/LSTM) → el reframe **con signo** → el **híbrido** → y finalmente **sweeps** de los finalistas. La conclusión es consistente y honesta:

Veredicto

Ningún camino superó al per-frame con clasificación explícita (model_01, F1 0.62/0.63). Los temporales no aprovechan la memoria (la mayoría de las etiquetas son interpoladas: el modelo aprendería el interpolador, no inercia real). El enfoque con signo es elegante pero rompe el STOP; el híbrido lo recupera pero no destrona al per-frame. El sweep, exhaustivo, confirma que el per-frame ya está en su techo: tunear no lo mueve. **El límite no es la arquitectura, es el dato** (91% de etiquetas interpoladas, una sola sesión, validación chica y de distribución corrida).

Y un aprendizaje metodológico que vale tanto como los modelos: con una validación tan chica, **las métricas son ruidosas**; el mecanismo de reseed mostró que varios “ganadores” de una sola semilla eran casualidad. Medir con rigor (Pareto + múltiples semillas) es parte del resultado. Para el informe, esto cumple de lleno el objetivo del enunciado de *discutir las limitaciones* del enfoque.